

Automation of Patient Specific Breast Cancer Image Segmentation Using 2D U-net MONAI

K S Suraksha Jain



Department of Applied Mechanics and Biomedical Engineering

Indian Institute of Technology Madras

Automation of Patient Specific Breast Cancer Image Segmentation Using 2D U-net MONAI

Dissertation submitted in partial fulfilment of the Requirement for the degree of

Master of Technology

In

Clinical Engineering

By

K S Suraksha Jain

(Roll NO: CL24M015)

Under the guidance of

Prof. Kavitha Arunachalam



June 2026

Department of Applied Mechanics and Biomedical Engineering

Indian Institute of Technology Madras

Declaration of Originality

I, K S Suraksha Jain, Roll Number CL24M015, hereby declare that this dissertation entitled **“Automation of Patient Specific Breast Cancer Image Segmentation Using 2D U-net MONAI”** represents my original work carried out at the Indian Institute of Technology Madras in partial fulfilment of the requirements for the degree of Master of Technology in Clinical Engineering. To the best of my knowledge, this dissertation contains no material previously published or written by another person, nor any material submitted for the award of any degree or diploma of this or any other institution.

Any contributions made to this research by others, with whom I have collaborated at IIT Madras or elsewhere, have been appropriately acknowledged in this dissertation. All sources of information and references used in this work have been duly cited under the appropriate sections of the dissertation.

I further declare that the work presented in this dissertation has been carried out under the guidance of Prof. Kavitha Arunachalam, Department of Engineering Design, IIT Madras, and that all experimental results, analyses, and interpretations presented are based on my original work unless otherwise stated.

I understand that in the event of any discrepancy or violation of academic integrity detected in the future, the Institute reserves the right to take appropriate action in accordance with its academic regulations.

June 1, 2026

IIT Madras

K S Suraksha Jain

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Prof. Kavitha Arunachalam, Department of Engineering Design, Indian Institute of Technology Madras (IIT Madras), for her constant guidance, valuable suggestions, encouragement, and support throughout the course of this project. Her insights and mentorship greatly helped in shaping the direction and successful completion of this work.

I would also like to thank all the faculty members and staff who provided the necessary support during the development of this project.

I extend my heartfelt gratitude to my lab mates, friends, and colleagues for their continuous encouragement, discussions, and support throughout the project. Their assistance and motivation made the entire development process both productive and enjoyable.

Finally, I would like to express my deepest gratitude to my parents and family for their unwavering support, patience, and encouragement at every stage of my academic journey. Their trust and motivation have always been a major source of strength and inspiration.

June 1, 2026
IIT Madras

K S Suraksha Jain
CL24M015

Abstract

Breast cancer remains one of the leading causes of cancer-related mortality among women worldwide, making early and accurate tumor localization essential for effective diagnosis and treatment planning. Manual delineation of breast tumours from Magnetic Resonance Imaging (MRI) is labour-intensive, time-consuming, and susceptible to inter-observer variability, thereby motivating the development of automated segmentation techniques.

This work presents an automated patient-specific breast tumor segmentation framework based on a two-dimensional (2D) U-Net architecture implemented using the Medical Open Network for AI (MONAI) framework. The proposed pipeline utilizes the Duke Breast Cancer MRI dataset and incorporates a comprehensive preprocessing strategy consisting of volume-level dataset splitting, voxel spacing standardization through resampling, axial slice extraction, and padding-based spatial standardization to generate uniform model inputs. In addition, multiple intensity normalization strategies, including Min–Max normalization, Min–Max normalization excluding background pixels, and Z-score normalization, were systematically evaluated to investigate their influence on segmentation performance.

The network was trained using a hybrid Dice Cross-Entropy loss function and evaluated using the Dice Similarity Coefficient (DSC), Intersection over Union (IoU), Precision, and Recall. Experimental results demonstrated that Z-score normalization achieved the highest validation Dice score of 0.6906, while Min–Max normalization yielded the highest independent test Dice score of 0.5702. To further improve segmentation quality, a two-stage post-processing strategy was introduced, consisting of the removal of small tumor predictions below 377.89 mm³ and elimination of anatomically isolated tumor regions using a median centroid-distance threshold of 63.69 mm. These post-processing operations effectively reduced false-positive detections while preserving clinically plausible tumor structures.

The proposed framework provides a computationally efficient and robust approach for automated breast MRI segmentation using a slice-based deep learning strategy, making it suitable for deployment on hardware with limited computational resources. Furthermore, the generated tumor segmentations can serve as a foundational component for future image-guided microwave-based breast cancer therapeutic systems.

Keywords: Breast Cancer, MRI, Medical Image Segmentation, U-Net, MONAI, Deep Learning, Intensity Normalization, Post-processing, NIfTI.

List of contents

Declaration of Originality	3
Acknowledgement	4
Abstract.....	5
1 Introduction	11
1.1 Background of Medical Imaging and MRI.....	11
1.2 Motivation for Breast Tumor Segmentation	13
1.3 Aim and Objective	14
1.4 Scope of this project.....	14
1.5 Proposed Methodology	15
1.6 Dataset Description	16
1.6.1 Train and Test Dataset Characteristics	18
2 Methodology	21
2.1 Overall Workflow	21
2.2 Data Preparation and Preprocessing	22
2.2.1 Loading and pairing of NIfTI Volumes	22
2.2.2 Volume-Level Dataset Preparation.....	24
2.2.3 Voxel Spacing Standardization Through Resampling.....	25
2.2.4 Slice Extraction.....	27
2.2.5 Padding-Based Spatial Standardization to 400 X 400	28
2.3 Model Training and Experimental Design.....	29
2.3.1 Dataset Loading and Organization	29
2.3.2 Transformation Pipeline	31
2.3.3 Data Augmentation	33
2.3.4 Intensity Normalization	34
2.3.5 Optimization Strategy and Training Procedure	37
2.4 Inference Procedure	39
2.5 Post-Processing	40
2.5.1 Removal of Small Tumor Components	41
2.5.2 Removal of Spatially Isolated Tumor Components.....	42
2.6 Results and Discussion	44

2.6.1	Overview of Experiments	44
2.6.2	Evaluation of Spatial Standardization through Resampling and Padding	44
2.6.3	Evaluation of Intensity Normalization Methods.....	49
2.7	Evaluation of Post-processing Strategy	53
3	Conclusion.....	57
4	Future work	58
5	References	59
	Appendix.....	61

Table of Figures

Figure 1 Comparison of DICOM and NIfTI data organization	11
Figure 2 Comparison of original breast MRI slices (left), lab-generated four-class segmentation used in this work as label (middle), and corresponding expert ground truth annotations (right) for selected test samples: (top) duke_009_0000.nii.gz, slice 78; (bottom) duke_022_0000.nii	17
Figure 3 Experimental Workflow for Breast MRI Segmentation Using 2D U-Net	21
Figure 4 Workflow of the breast MRI preprocessing pipeline for 2D segmentation model training.....	22
Figure 5 Folder structure of the automated breast segmentation project.....	23
Figure 6 Effect of voxel spacing variation on anatomical pixel representation across patients. Although both tumors have the same physical size, differences in voxel spacing cause them to occupy different numbers of pixels, demonstrating the need for voxel standardization through resampling	27
Figure 7 Overview of the training and validation transformation pipelines used for MRI slice preprocessing prior to U-Net training.....	32
Figure 8 Training dataset augmentation workflow implemented using MONAI transforms. Random rotations and flipping operations were applied online to both MRI slices and corresponding segmentation masks before conversion to PyTorch tensor format and input to the 2D U-Net Model.....	34
Figure 9 Experimental intensity normalization configurations evaluated in this study, including no normalization, Min–Max normalization (with and without background), and Z-score normalization (with background) applied during the MRI preprocessing pipeline	35
Figure 10 Inference workflow of the trained 2D U-Net model. The input MRI slice is processed through the trained network to produce a pixel-wise multi-class segmentation mask	40
Figure 11 Graphical representation of the distribution of ground-truth tumor volumes across the 40 annotated MRI cases arranged in descending order.	41
Figure 12 Visual comparison of representative breast MRI slices before and after voxel spacing standardization through resampling. The observed changes correspond to image dimensions (pixel	

representation) resulting from voxel redistribution and do not indicate any physical change in breast size or tumor morphology.....	47
Figure 13 Representative breast MRI slices before and after padding to 400×400 pixels.	49
Figure 14 Qualitative comparison of segmentation results obtained using different intensity normalization methods for the first five representative test MRI volumes. The predicted tumor regions are overlaid on the corresponding breast MRI volumes, illustrating the effect of each normalization strategy on segmentation performance.	51
Figure 15 Qualitative comparison of segmentation results obtained using different intensity normalization methods for the remaining five representative test MRI volumes. The visualization further demonstrates the differences in segmentation quality produced by the evaluated normalization approaches.	52
Figure 16 Illustration of the two-stage post-processing strategy. First being (a) Elimination of predicted tumor regions with volumes below 377.89 mm^3	55
Figure 17 . Illustrates second stage of postprocessing (b) Removal of secondary tumor regions whose centroid distance from the primary tumor exceeds the median distance threshold.....	56

List of Tables

Table 1-Class labels and corresponding color scheme adopted in this study for multi-class breast MRI segmentation	18
Table 2-Characteristics of the Train Dataset Including Volume Dimensions and Voxel Spacing	18
Table 3-Characteristics of the Test Dataset Including Volume Dimensions and Voxel Spacing	20
Table 4-Comparison of data loading times obtained using different numbers of DataLoader worker processes	31
Table 5 Analysis of tumor region volumes and centroid distances used for distance-based post-processing.	43
Table 6 summarizes the effect of the resampling operation on the 30 MRI volumes included in the training dataset. The original in-plane voxel spacing varied from approximately 0.5859 mm to 1.0625 mm because of differences in imaging protocols. Following	44
Table 7 Original and resampled in-plane image sizes and voxel spacings of the test MRI dataset.	48
Table 8 Quantitative comparison of different intensity normalization methods based on the best validation Dice score and independent test-set segmentation metrics.....	50
Table 9 Distribution of ground-truth tumor volumes across the 40 annotated MRI cases arranged in descending order	53

“Dedicated to the Almighty, my Supervisor Prof. Kavitha Arunachalam, my
parents, my lab mates, my friends”

1 Introduction

1.1 Background of Medical Imaging and MRI

Medical imaging plays a crucial role in modern healthcare by enabling non-invasive visualization of internal anatomical structures. It allows clinicians to diagnose diseases, monitor progression, and plan treatments without the need for surgical intervention. Among the various imaging modalities, Magnetic Resonance Imaging (MRI) is widely used due to its superior soft-tissue contrast and its ability to generate high-resolution three-dimensional representations of organs and tissues. This makes MRI particularly suitable for applications such as tumor detection and analysis, where clear differentiation between tissues is essential.

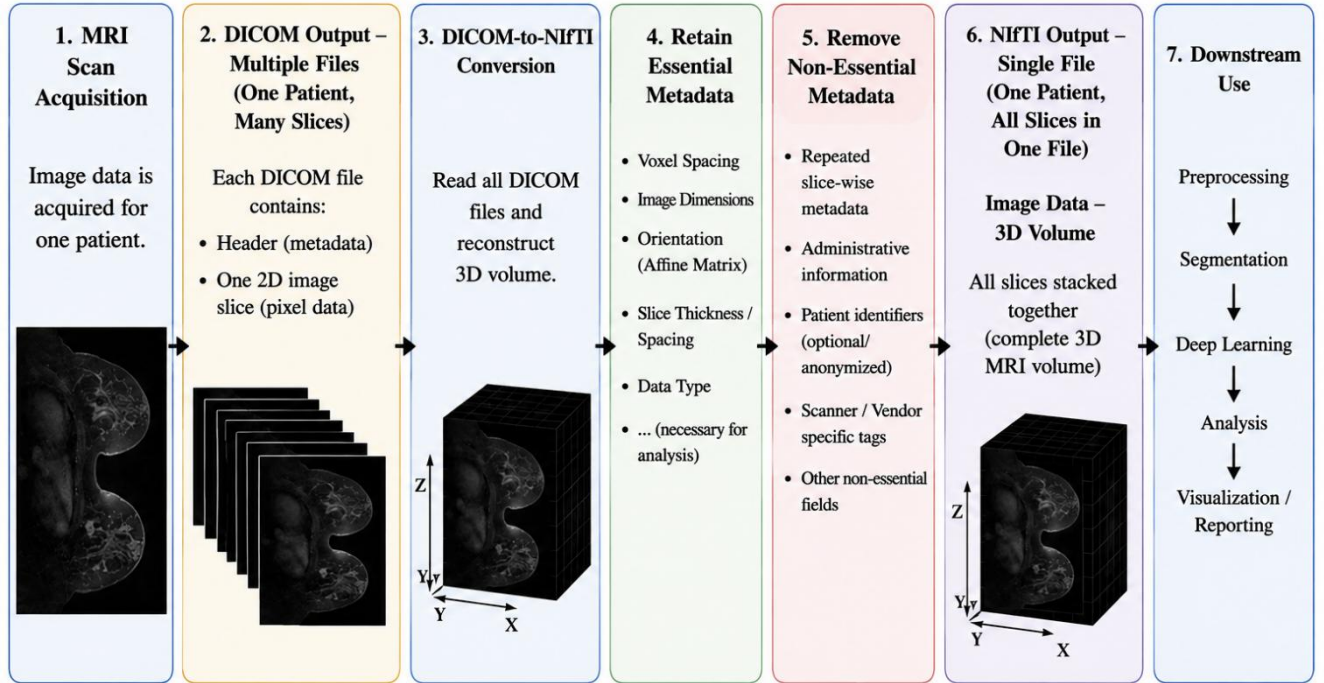


Figure 1 Comparison of DICOM and NIfTI data organization

MRI data is commonly stored in the Neuroimaging Informatics Technology Initiative (NIfTI) format (i.e., *.nii* or *.nii.gz*), which is specifically designed for volumetric medical imaging and computational analysis. Unlike traditional image formats, NIfTI stores the entire three-dimensional MRI volume in a single file while preserving important spatial metadata such as voxel dimensions (spacing),

image dimensions, and anatomical orientation. In contrast, the Digital Imaging and Communications in Medicine (DICOM) format stores medical images as multiple individual slice files, where each file typically contains a metadata header and one two-dimensional (2D) image slice. The image data represents the actual grayscale anatomical scan, whereas the header is a structured section within the file that stores information describing how the image should be interpreted and reconstructed. This metadata may include patient and study information, slice thickness, pixel spacing, image orientation, scanner parameters, and acquisition settings. The textual information displayed around medical images in viewing software, such as orientation markers and acquisition parameters, is derived from this metadata stored in the DICOM header. Since each DICOM slice contains its own header, many metadata fields are repeatedly stored across hundreds of slice files for a single patient scan, resulting in increased redundancy and storage complexity.

As illustrated in Figure 1, the MRI acquisition process initially produces multiple DICOM files for a single patient, where each file contains one image slice and its associated metadata header. During DICOM-to-NIfTI conversion, all DICOM slice files are read and reconstructed into a complete three-dimensional (3D) volume. In this process, only the essential metadata required for spatial reconstruction and computational analysis, such as voxel spacing, image dimensions, orientation (affine matrix), slice spacing, and data type, are retained within a simplified NIfTI header. Non-essential or redundant metadata, including repeated slice-wise information, administrative details, patient identifiers, and vendor-specific tags, are typically removed. Consequently, the resulting NIfTI file contains a single common header for the entire volumetric scan along with all reconstructed image slices stored together within one file.

The NIfTI format therefore provides a more streamlined and unified representation, making it highly suitable for deep learning-based medical image analysis. Its single-file volumetric structure facilitates efficient data loading, preprocessing, and manipulation, which are essential for training segmentation models. This is particularly beneficial for both two-dimensional (2D) and three-dimensional (3D) U-Net architectures. In 2D approaches, volumetric data can be easily decomposed into consistent slices, while in 3D approaches, entire volumes can be processed directly without requiring additional reconstruction. Furthermore, unlike standard image formats such as JPEG or PNG, NIfTI preserves the full volumetric structure and numerical precision of the imaging data, which is critical for accurate medical image segmentation and quantitative analysis tasks.

1.2 Motivation for Breast Tumor Segmentation

Breast cancer is one of the most prevalent cancers worldwide and remains a major cause of morbidity and mortality among women [1]. According to the latest estimates from the World Health Organization (WHO) and the Global Cancer Observatory (GLOBOCAN), approximately 2.3 million women were diagnosed with breast cancer in 2022, accounting for 11.6% of all new cancer cases globally, while an estimated 670,000 deaths were attributed to the disease [2]. Furthermore, breast cancer is the most commonly diagnosed cancer among women in 157 out of 185 countries, highlighting its significant global health burden. Early diagnosis and accurate assessment of tumor characteristics are therefore essential for improving survival rates and treatment effectiveness.

Accurate localization and delineation of tumor boundaries are critical components of diagnosis and treatment planning. Precise segmentation allows clinicians to estimate tumor size, shape, and spatial distribution more effectively, which directly influences therapeutic decision-making. Manual segmentation performed by experts is often time-consuming, labor-intensive, and subject to inter-observer variability. As a result, automated and reliable segmentation methods have gained significant importance in recent years to improve consistency, reduce clinical workload, and support large-scale analysis.

The motivation for this work originates from a microwave-based therapeutic device developed in the laboratory for targeted breast tumor treatment. The device is designed to selectively heat tumor tissues while minimizing thermal exposure to surrounding healthy tissue. This targeted heating approach depends heavily on accurate identification of tumor location and boundaries, since localization errors may result in inadequate treatment delivery or unintended damage to normal tissue structures. Therefore, obtaining precise tumor segmentation becomes an essential prerequisite for the effective functioning of the therapeutic system.

To enable this targeted therapeutic strategy, the proposed framework utilizes breast MRI data as input for automated tumor segmentation. The MRI scans are processed to identify and isolate tumor regions with minimal manual intervention. These segmented regions can subsequently be incorporated into treatment planning by guiding controlled heating over the actual tumor location. By establishing a connection between medical image analysis and therapeutic intervention, the proposed workflow aims to improve treatment precision and efficient clinical care. This work focuses on developing a

computationally efficient segmentation framework that remains compatible with lower-end hardware configurations. By adopting a slice-based 2D processing strategy.

1.3 Aim and Objective

Aim

To develop an efficient and accurate breast tumor segmentation framework using a 2D U-Net model implemented in MONAI, by incorporating appropriate preprocessing, normalization and postprocessing strategies to improve segmentation performance while maintaining computational efficiency.

Objectives

- To preprocess breast MRI data by converting volumetric NIfTI images into 2D slices suitable for model training.
- To apply spatial preprocessing techniques including resampling, padding.
- To implement a 2D U-Net architecture using the MONAI framework for automated breast tumor segmentation.
- To evaluate and compare different data normalization techniques, including Min–Max normalization and Z-score normalization, to determine their impact on segmentation performance.
- To analyze the effect of including and excluding background regions during normalization on model accuracy and stability.
- To develop an efficient segmentation pipeline that balances computational cost and performance by leveraging 2D slice-based processing instead of full 3D modeling.
- To ensure that the segmented tumor regions maintain spatial consistency for accurate localization in clinical applications.

1.4 Scope of this project

This project focuses on the development of an automated patient-specific breast tumor segmentation framework using a 2D U-Net architecture implemented in the MONAI framework [3] [4]. The work covers the complete segmentation pipeline, including NIfTI data loading and organization,

volume-level dataset splitting, voxel spacing standardization through resampling, axial slice extraction, padding-based spatial standardization, intensity normalization, data augmentation, model training, inference, quantitative evaluation, and post-processing using connected component analysis.

The study utilizes the Duke Breast Cancer MRI dataset, comprising 30 MRI volumes for training and 10 MRI volumes for independent testing. Volume-level splitting was performed before slice extraction, resulting in approximately 24 training volumes and 6 validation volumes. After preprocessing, all axial slices were retained, producing approximately 5,000 training slices, 1,100 validation slices, and 1,778 testing slices. Model performance was monitored using the Validation Dice Similarity Coefficient (DSC), while the final test dataset was evaluated using Dice Similarity Coefficient (DSC), Intersection over Union (IoU), Precision, and Recall.

The framework was developed in an isolated Miniconda environment using Python 3.10.19, PyTorch 2.2.2, and MONAI 1.5.1, with GPU acceleration provided by an NVIDIA T1000 (4 GB VRAM) running CUDA 11.8. Major supporting libraries include NumPy, SimpleITK, NiBabel, SciPy, OpenCV, Matplotlib, Pandas, and Scikit-learn. The proposed framework provides an efficient automated breast MRI segmentation and future integration with image-guided therapeutic applications.

1.5 Proposed Methodology

In this work, an efficient deep learning-based approach is proposed for breast tumor segmentation from MRI data [5]. Unlike traditional manual segmentation methods, which are time-consuming and subject to inter-observer variability, the proposed method utilizes an automated framework to achieve consistent and reliable results.

Direct application of deep learning models on volumetric (3D) MRI data is computationally intensive and requires significant memory resources. To address this challenge, a slice-based approach is adopted, where 3D MRI volumes are decomposed into 2D axial slices. This reduces computational complexity while preserving essential anatomical information.

The proposed framework employs a 2D U-Net architecture implemented using the MONAI framework. Prior to model training, appropriate preprocessing and normalization strategies are applied to the MRI data to enhance model performance and ensure stable training. The processed slices are then used as input to the network for segmentation.

This approach enables efficient handling of medical imaging data while maintaining segmentation accuracy, making it suitable for applications requiring precise tumor localization.

1.6 Dataset Description

The study titled “*A Large-Scale Multicenter Breast Cancer DCE-MRI Benchmark Dataset with Expert Segmentations*” combines multiple publicly available breast DCE-MRI datasets to create a comprehensive benchmark for breast tumor segmentation research [6]. The benchmark incorporates imaging data from Duke Breast Cancer MRI (DUKE), I-SPY1, I-SPY2, and NACT-Pilot, providing diversity in acquisition protocols, scanner configurations, patient populations, and imaging characteristics. This multicenter design improves dataset variability and supports the development of segmentation models with better clinical generalization. Among these available datasets from the paper this work utilizes the Duke Breast Cancer MRI dataset due to its favorable imaging characteristics, including complete bilateral breast imaging, relatively thin slice thickness (approximately 1.1 mm), higher spatial resolution, and preservation of detailed anatomical structures. The dataset includes expert-derived tumor annotations that serve as ground truth labels for supervised training of the proposed multi-class segmentation framework. These expert-provided segmentations represent clinically validated tumor delineations and were used to generate the corresponding four-class segmentation labels adopted in this study. Compared with a conventional binary tumor–background representation, the use of four segmentation classes provides richer structural information and enables the model to learn distinct characteristics across different tissue regions, supporting more detailed tumor characterization. In addition, the original expert segmentation masks can be used as reference standards for direct visual comparison(qualitative) with the predicted segmentation outputs, also quantitative assessment of how closely the model reproduces expert-defined tumor boundaries.

The anatomical orientation metadata of all MRI volumes was inspected using the affine transformation matrix stored in the NIfTI header through the `nibabel.aff2axcodes()` function. The affine transformation matrix defines the spatial orientation, voxel spacing, scaling, and physical positioning of the image volume within real-world anatomical space. All training and test MRI volumes were found to follow a consistent anatomical orientation of (L, A, I), corresponding to Left–Anterior–Inferior orientation. Maintaining a uniform anatomical orientation across all volumes is beneficial because it ensures that the same anatomical structures appear in consistent spatial locations across subjects. This reduces preprocessing complexity and helps prevent left–right or superior–inferior

misalignment errors during volumetric reconstruction, preprocessing, and deep learning-based segmentation.

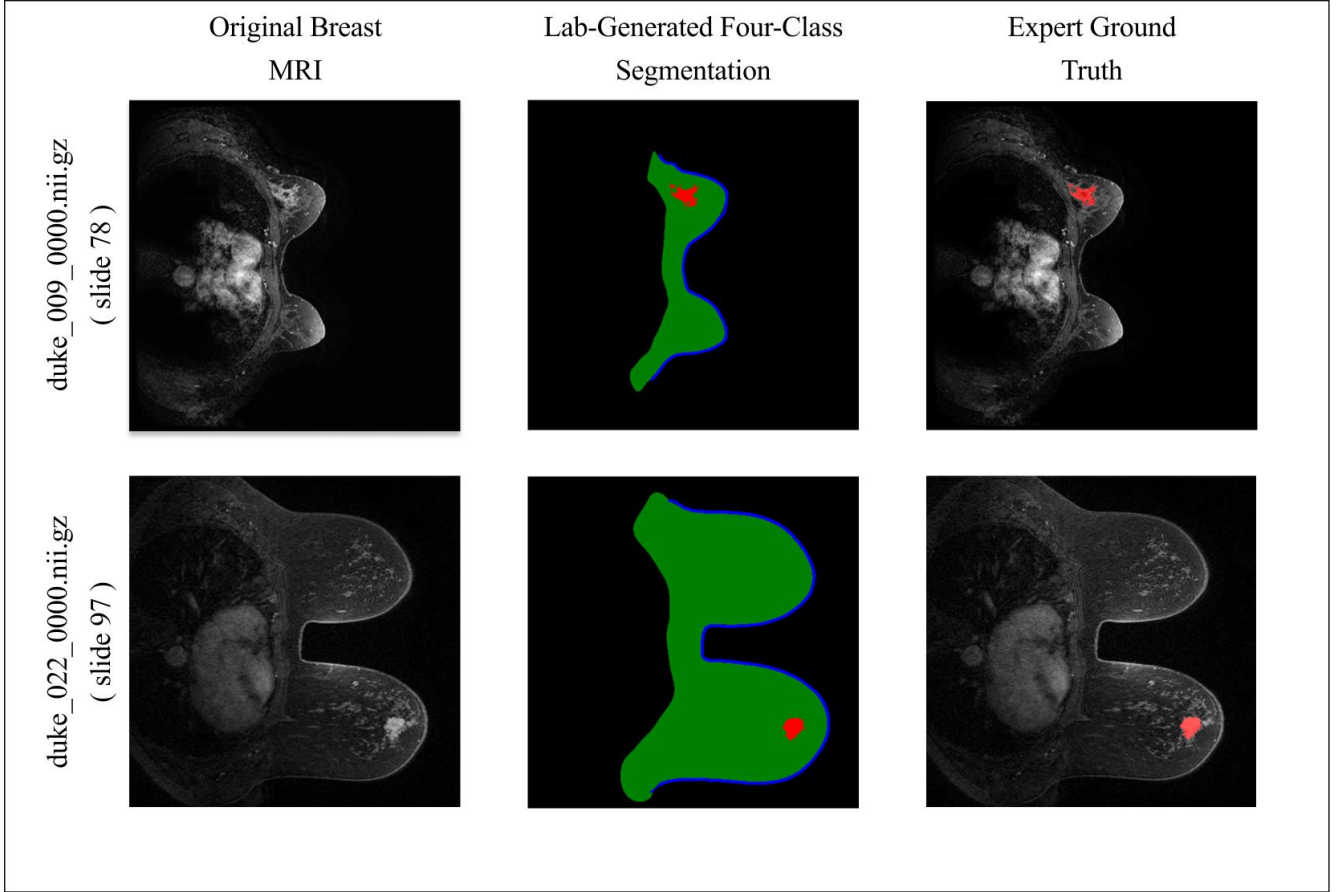


Figure 2 Comparison of original breast MRI slices (left), lab-generated four-class segmentation used in this work as label (middle), and corresponding expert ground truth annotations (right) for selected test samples: (top) duke_009_0000.nii.gz, slice 78; (bottom) duke_022_0000.nii

In the dataset, each patient is represented using a unique identifier following the naming convention duke_XXX (e.g., duke_034_0001.nii.gz), where duke_034 corresponds to an individual patient. The suffixes _0000, _0001, _0002, and subsequent indices denote different temporal phases of the Dynamic Contrast-Enhanced Magnetic Resonance Imaging (DCE-MRI) acquisition. DCE-MRI is an imaging technique in which sequential MRI scans are acquired before and after administration of a contrast agent to evaluate how the contrast material enters and leaves tissues over time, thereby enabling assessment of tissue perfusion and tumor enhancement characteristics. In this dataset, _0000 represents the pre-contrast MRI volume acquired before contrast-agent administration, while the subsequent phases correspond to post-contrast acquisitions captured at different time points to monitor tissue and tumor

enhancement patterns over time. A key advantage of this dataset is its multicenter nature, as the data is collected from multiple medical institutions with different devices. This introduces variability in imaging conditions, scanner types, and patient populations, making the dataset more representative of real-world clinical scenarios and improving the generalization capability of the developed model.

Table 1-Class labels and corresponding color scheme adopted in this study for multi-class breast MRI segmentation

Class Label	Assigned Label ID (Color)
Background	0 (Black)
Tumor	1 (Red)
Glandular tissues	2 (Green)
Skin	3 (Blue)

In work uses label which is extended to include other anatomical structures unlike the original tumor-only annotations from the paper “*A Large-Scale Multicenter Breast Cancer DCE-MRI Benchmark Dataset with Expert Segmentations*” like shown in Table 1, which includes four distinct classes: background, tumor, glandular tissue, and skin [7]. The corresponding class labels together with their assigned label IDs and color scheme adopted for visualization presented in Figure 2.

This multi-class labeling provides richer anatomical context and enables improved visualization of tumor location and its relationship with surrounding breast structures compared with the original tumor-only segmentation representation.

1.6.1 Train and Test Dataset Characteristics

Table 2-Characteristics of the Train Dataset Including Volume Dimensions and Voxel Spacing

Sample ID	Volume Dimension (H x W x D)	Voxel Spacing (mm))
duke_001_0000.nii.gz	(448, 448, 160)	(0.8036, 0.8036, 1.1000)
duke_002_0000.nii.gz	(512, 512, 142)	(0.5859, 0.5859, 1.3000)
duke_005_0000.nii.gz	(448, 448, 160)	(0.7813, 0.7813, 1.0000)
duke_009_0000.nii.gz	(512, 512, 156)	(0.6641, 0.6641, 1.1000)
duke_010_0000.nii.gz	(512, 512, 146)	(0.6641, 0.6641, 1.0000)
duke_019_0000.nii.gz	(320, 320, 160)	(1.0625, 1.0625, 1.1000)
duke_021_0000.nii.gz	(512, 512, 176)	(0.7422, 0.7422, 1.0000)
duke_022_0000.nii.gz	(448, 448, 160)	(0.7366, 0.7366, 1.1000)

duke_034_0001.nii.gz	(512, 512, 148)	(0.7031, 0.7031, 1.2000)
duke_040_0001.nii.gz	(448, 448, 160)	(0.7589, 0.7589, 1.1000)
duke_041_0001.nii.gz	(512, 512, 168)	(0.6836, 0.6836, 1.0000)
duke_043_0001.nii.gz	(512, 512, 188)	(0.6836, 0.6836, 1.0000)
duke_044_0001.nii.gz	(448, 448, 128)	(0.7589, 0.7589, 1.1000)
duke_046_0001.nii.gz	(448, 448, 256)	(0.8371, 0.8371, 1.0000)
duke_048_0001.nii.gz	(448, 448, 176)	(0.7589, 0.7589, 1.0000)
duke_051_0001.nii.gz	(512, 512, 200)	(0.7422, 0.7422, 1.0000)
duke_055_0001.nii.gz	(512, 512, 170)	(0.6445, 0.6445, 1.0000)
duke_057_0001.nii.gz	(448, 448, 112)	(0.8036, 0.8036, 1.5000)
duke_059_0001.nii.gz	(512, 512, 182)	(0.6641, 0.6641, 1.0000)
duke_060_0001.nii.gz	(512, 512, 178)	(0.6836, 0.6836, 1.0000)
duke_061_0001.nii.gz	(512, 512, 174)	(0.6641, 0.6641, 1.0000)
duke_064_0001.nii.gz	(512, 512, 162)	(0.6445, 0.6445, 1.0000)
duke_069_0001.nii.gz	(512, 512, 196)	(0.6641, 0.6641, 1.0000)
duke_082_0001.nii.gz	(448, 448, 176)	(0.7589, 0.7589, 1.1000)
duke_086_0001.nii.gz	(448, 448, 208)	(0.7813, 0.7813, 1.0000)
duke_090_0001.nii.gz	(512, 512, 196)	(0.6836, 0.6836, 1.0000)
duke_091_0001.nii.gz	(448, 448, 160)	(0.7589, 0.7589, 1.1000)
duke_097_0001.nii.gz	(448, 448, 208)	(0.7813, 0.7813, 1.0000)
duke_099_0001.nii.gz	(512, 512, 148)	(0.6250, 0.6250, 1.0000)
duke_101_0001.nii.gz	(448, 448, 160)	(0.6920, 0.6920, 1.1000)

The train dataset as shown in Table 2 exhibits variability in both volume dimensions and voxel spacing, reflecting differences in MRI acquisition settings across patient scans. Volume dimensions are represented as Width $\times$ Height $\times$ Depth ($W \times H \times D$), where H and W indicate the pixel dimensions of each MRI slice and D represents the total number of slices in the volume. For example, $(512 \times 512 \times 142)$ indicates a volume containing 142 slices, each of size 512×512 pixels. Voxel spacing is represented as (X, Y, Z) in millimeters, where X and Y denote spatial resolution within a slice and Z denotes spacing between slices. In the selected train dataset, the MRI volumes exhibited in-plane dimensions ranging from 320×320 to 512×512 , while the number of slices varied from 112 to 256 across different patients. The voxel spacing ranged from 0.5859 mm to 1.0625 mm in-plane and from 1.0 mm to 1.5 mm between slices. These variations influence image appearance and are addressed through preprocessing and normalization to ensure consistent model inputs. Overall, 30 MRI volumes were selected as the training

dataset, providing representative variability in volume dimensions and voxel spacing for model training and improving robustness to differences in acquisition settings.

The test dataset also exhibits variability in both volume dimensions and voxel spacing, reflecting differences in MRI acquisition settings across patient scans, as summarized in Table 3. The MRI volume dimensions varied from 448×448 to 512×512 in-plane resolution, while the number of slices ranged from 142 to 208 across different patients. The voxel spacing varied from 0.625 mm to 0.8036 mm in-plane and from 1.0 mm to 1.1 mm between slices.

Table 3-Characteristics of the Test Dataset Including Volume Dimensions and Voxel Spacing

Sample ID	Volume Dimension (H x W x D)	Voxel Spacing (mm))
duke_470_0001.nii	(512, 512, 200)	(0.7227, 0.7227, 1.1)
duke_508_0001.nii	(512, 512, 174)	(0.7031, 0.7031, 1.0)
duke_512_0001.nii	(448, 448, 160)	(0.7812, 0.7812, 1.0)
duke_514_0001.nii	(512, 512, 174)	(0.7031, 0.7031, 1.0)
duke_517_0001.nii	(512, 512, 142)	(0.6836, 0.6836, 1.0)
duke_521_0001.nii	(512, 512, 174)	(0.625, 0.625, 1.0)
duke_525_0001.nii	(512, 512, 196)	(0.7422, 0.7422, 1.0)
duke_527_0001.nii	(448, 448, 208)	(0.8036, 0.8036, 1.0)
duke_530_0001.nii	(512, 512, 174)	(0.6641, 0.6641, 1.0)
duke_534_0001.nii	(448, 448, 176)	(0.8036, 0.8036, 1.0)

2 Methodology

2.1 Overall Workflow

The overall workflow begins with paired breast MRI volumes and their corresponding modified ground truth masks containing four segmentation classes: background, tumor, glandular tissue, and skin. The MRI volumes and labels are first loaded, organized, and standardized through voxel spacing resampling to ensure consistent physical representation across patients. The standardized volumes are then converted into axial two-dimensional slices, followed by padding-based spatial standardization to

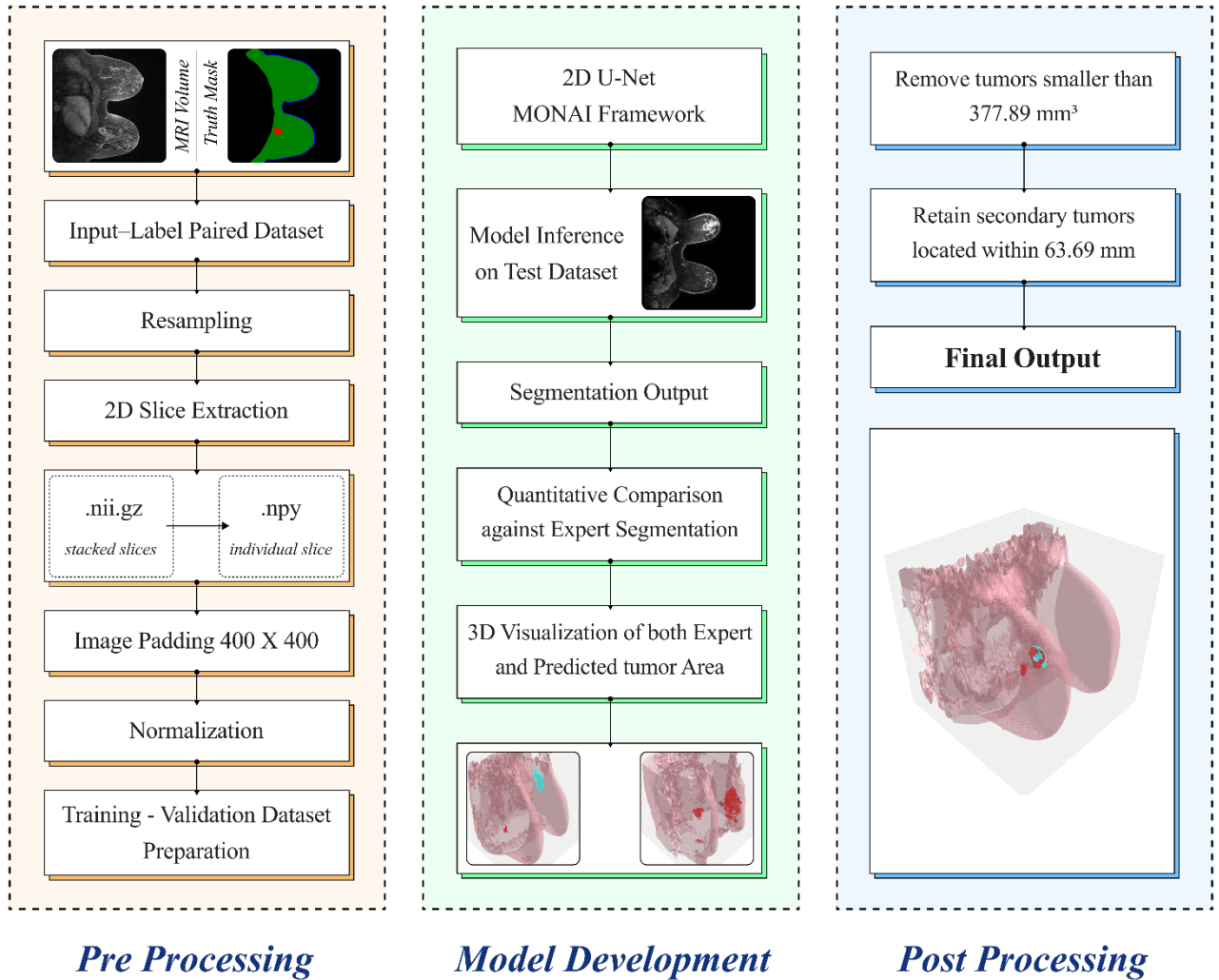


Figure 3 Experimental Workflow for Breast MRI Segmentation Using 2D U-Net

generate uniform input dimensions. A volume-wise train-validation split is subsequently performed to

prevent data leakage between patients, after which the slices undergo intensity normalization and data augmentation. The processed image–label pairs are then used to train a MONAI-based 2D U-Net model for multi-class segmentation. During inference, previously unseen test MRI volumes are subjected to the same preprocessing pipeline and provided to the trained network to generate segmentation masks. Finally, the predicted outputs are quantitatively evaluated and refined using post-processing operations to reduce false-positive detections and improve anatomical consistency.

2.2 Data Preparation and Preprocessing

To prepare the breast MRI dataset for two-dimensional deep learning–based segmentation, a structured preprocessing pipeline was implemented. The pipeline consisted of loading and pairing MRI volumes with their corresponding segmentation masks, performing volume-level train–validation

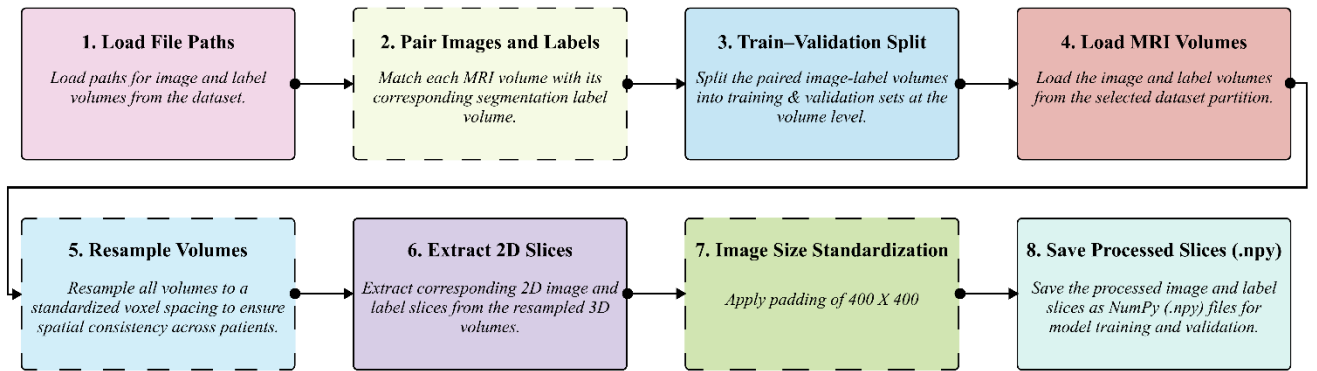


Figure 4 Workflow of the breast MRI preprocessing pipeline for 2D segmentation model training.

splitting, standardizing voxel spacing through resampling, extracting two-dimensional axial slices, and applying spatial standardization strategy to generate fixed-size model inputs - padding-based standardization to 400 X 400 pixels. Finally, the processed image and label slices were stored as NumPy arrays for efficient loading during model training and evaluation. The overall preprocessing workflow is illustrated in Figure 4.

2.2.1 Loading and pairing of NIfTI Volumes

The MRI volumes and their corresponding labels were stored separately in NIfTI format within dedicated image (imagesTr) and label (labelsTr) directories, as shown in Figure 5. To enable correct

image-label association, consistent file naming was maintained at the dataset level, where corresponding MRI volumes and labels shared the same file/patient identifier (e.g., duke_001 in duke_001_0000.nii.gz and duke_001.nii.gz). During data loading, file paths from the image and label directories were retrieved separately using the `glob.glob()` function from Python's glob module. The function searched the specified directories for files matching the compressed NIfTI (`.nii.gz`) pattern and returned their file paths as a list, enabling subsequent preprocessing and pairing operations to be applied systematically. The resulting file lists were then arranged using Python's built-in `sorted()` function to maintain identical ordering across image and label datasets and ensure accurate image-label association. The ordered file lists were then paired using Python's `zip()` function, allowing each MRI volume to be automatically matched with its corresponding ground truth label. The ordered file lists were then paired using Python's `zip()` function, which generated corresponding image-label pairs as tuples, ensuring that each MRI volume was matched with its associated ground truth label.

The paired NIfTI volumes were subsequently loaded using the `nib.load()` function from the NiBabel library, which creates a NIfTI image object containing the image metadata and a reference to the stored image data. The actual voxel intensity values were then retrieved using the NiBabel `get_fdata()` method, which reads the image data section of the NIfTI file and converts it into a NumPy array for subsequent preprocessing and analysis. Thus, `glob.glob()` was used only to identify and

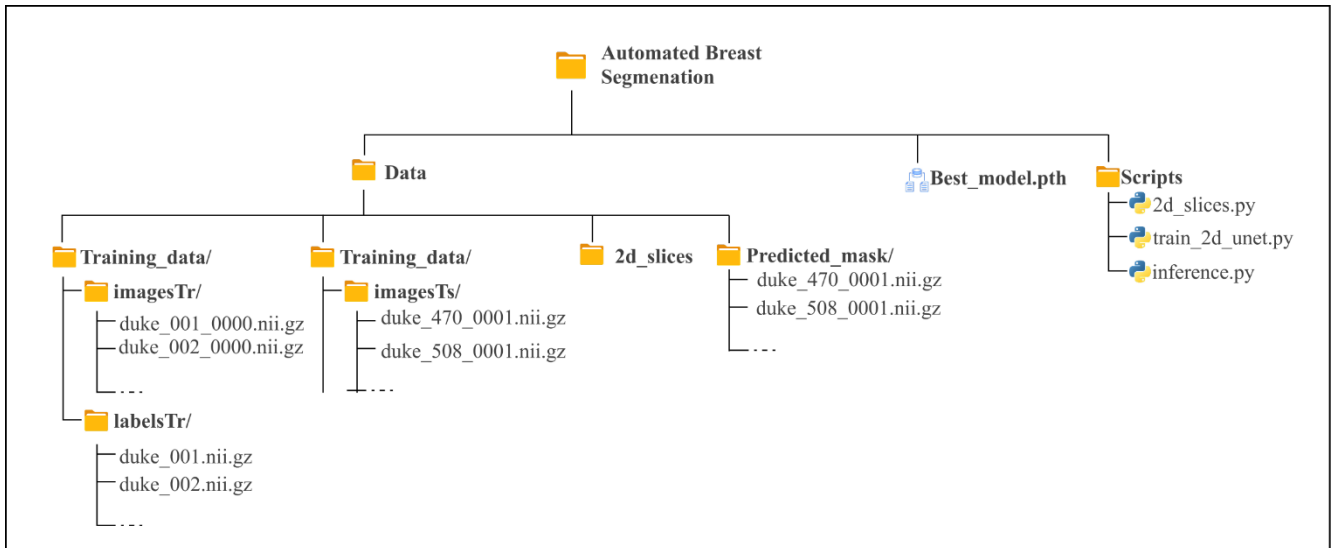


Figure 5 Folder structure of the automated breast segmentation project.

list the file paths of NIfTI volumes, whereas `nib.load()` was used to actually open and read the contents of those files, enabling access to both the volumetric image data and associated metadata.

These loading and pairing procedures prevented image-label mismatches and ensured that all subsequent preprocessing operations, including resampling, spatial standardization, normalization, and slice extraction, were applied consistently to both MRI volumes and segmentation masks. Maintaining this one-to-one anatomical correspondence preserved spatial accuracy throughout preprocessing and supported reliable segmentation model training and evaluation.

The implementation of the loading and pairing procedure is provided in Appendix under the file `2d_slicing.py`, in the section titled “Load volumes.”

2.2.2 Volume-Level Dataset Preparation

To support reliable model generalization and meaningful performance evaluation, special care was taken during dataset preparation because the MONAI-based 2D U-Net framework processes individual 2D slices rather than complete 3D MRI volumes as input. Therefore, dataset splitting was performed at the volume level before slice extraction, where each complete MRI volume and its corresponding ground truth label were treated as a single unit during the splitting process. During implementation, paired MRI volumes and labels were first grouped together and then divided into training and validation sets using the `train_test_split()` function before any slice-wise processing was performed.

This ensured that all slices originating from a given patient volume were assigned entirely to either the training set or the validation set, but never distributed across both sets. Volume-level splitting is beneficial because adjacent slices within an MRI volume are anatomically continuous and highly similar, making them strongly correlated rather than independent samples. If slice extraction had been performed before splitting, slices from the same patient could appear in both datasets, allowing the model to encounter nearly identical anatomical structures during training and evaluation. Such overlap could unintentionally encourage the model to learn patient-specific features rather than generalized segmentation patterns and would lead to artificially optimistic validation performance.

By preserving patient-level separation prior to slice generation, the validation set better represented truly unseen data and enabled a more realistic assessment of segmentation performance. This strategy also ensured that subsequent preprocessing operations, including resampling, padding, and slice

extraction, were applied independently within each dataset partition, improving the reliability and validity of model evaluation. The implementation of volume-level splitting is provided in Appendix , under the file 2d_slicing.py, in the section titled “Train/Val split (volume level).”

2.2.3 Voxel Spacing Standardization Through Resampling

The acquired breast MRI volumes exhibited variations in voxel spacing across patients due to differences in acquisition settings and scanner configurations, as summarized in Table 2 and Table 3. In medical imaging, voxel spacing defines the real-world physical size represented by each pixel or voxel within the image and is typically expressed in millimeters along the X, Y, and Z directions. Consequently, identical anatomical structures may occupy different numbers of pixels across patients despite having the same physical dimensions. The relationship between physical dimensions and image in be expressed as:

$$L = N \times S \quad i$$

where L represents the physical length (mm), N represents the number of pixels or voxels, and S represents the voxel spacing (mm/voxel) in equation i. Consequently, identical anatomical structures may occupy different numbers of pixels across patients despite having the same physical dimensions. For example, a tumor with an identical real-world size may appear larger in pixel representation in a high-resolution image with smaller voxel spacing (e.g., 0.5 mm $\times$ 0.5 mm) and smaller in pixel representation in a lower-resolution image with larger voxel spacing (e.g., 1.0 mm $\times$ 1.0 mm). Therefore, direct comparison of raw pixel dimensions across patients becomes unreliable without voxel spacing standardization

Since the implemented 2D U-Net architecture operates on pixel-based image inputs and does not directly utilize physical spacing information stored within the NIfTI metadata, voxel spacing standardization was performed prior to slice extraction and model training. Resampling was performed before slice extraction to ensure that all extracted slices originated from MRI volumes with standardized voxel spacing, thereby providing a consistent anatomical scale across the dataset before generation of the 2D training samples. Without prior resampling, extracted 2D slices would initially retain differences in anatomical pixel representation caused by inconsistent voxel spacing across patients, where physically

similar structures could occupy different numbers of pixels. By performing volumetric resampling before slice extraction, anatomically similar structures were represented at comparable physical scales across the dataset, thereby improving spatial consistency and ensuring that all subsequently extracted slices originated from standardized anatomical representations.

Resampling was implemented using SciPy's `ndimage.zoom()` function. Resize factors were computed as the ratio between the original voxel spacing and the target voxel spacing. MRI intensity volumes were resampled using linear interpolation (`order = 1`) because MRI images contain continuous intensity values that represent tissue characteristics. During resampling, new voxel locations are created and their intensity values must be estimated from neighboring voxels. Linear interpolation calculates these values using surrounding intensity information, thereby producing smooth transitions and preserving anatomical continuity while minimizing abrupt intensity changes. The primary purpose of resampling is to standardize the voxel spacing across all MRI volumes so that each pixel represents the same physical distance. Since convolutional neural networks operate on pixel intensities rather than real-world measurements, variations in voxel spacing can cause identical anatomical structures to appear at different scales. By resampling all images to a common in-plane resolution of $1 \times 1 \text{ mm}^2$, anatomical features are represented consistently across the dataset, thereby improving the robustness and generalizability of the segmentation model.

In contrast, segmentation masks were resampled using nearest-neighbor interpolation (`order = 0`) because they contain discrete categorical labels rather than continuous intensity values. Each pixel in a segmentation mask represents a specific anatomical class (e.g., background or tumor), and introducing intermediate values through linear interpolation would generate invalid label values and distort segmentation boundaries. Nearest-neighbor interpolation preserves the original class assignments by selecting the label of the closest voxel, thereby maintaining discrete class boundaries and preventing artificial mixing of segmentation labels.

Resampling was intentionally performed before padding because the two operations serve different purposes. Resampling standardizes the voxel spacing, ensuring that anatomical structures are represented at a consistent physical scale across all MRI volumes. Padding, on the other hand, only adjusts the image matrix dimensions required for deep learning model input by adding background pixels around the image. Performing resampling first prevents unnecessary interpolation of padded regions and allows fixed-size padding to be applied only after all images have been standardized to a common voxel spacing.

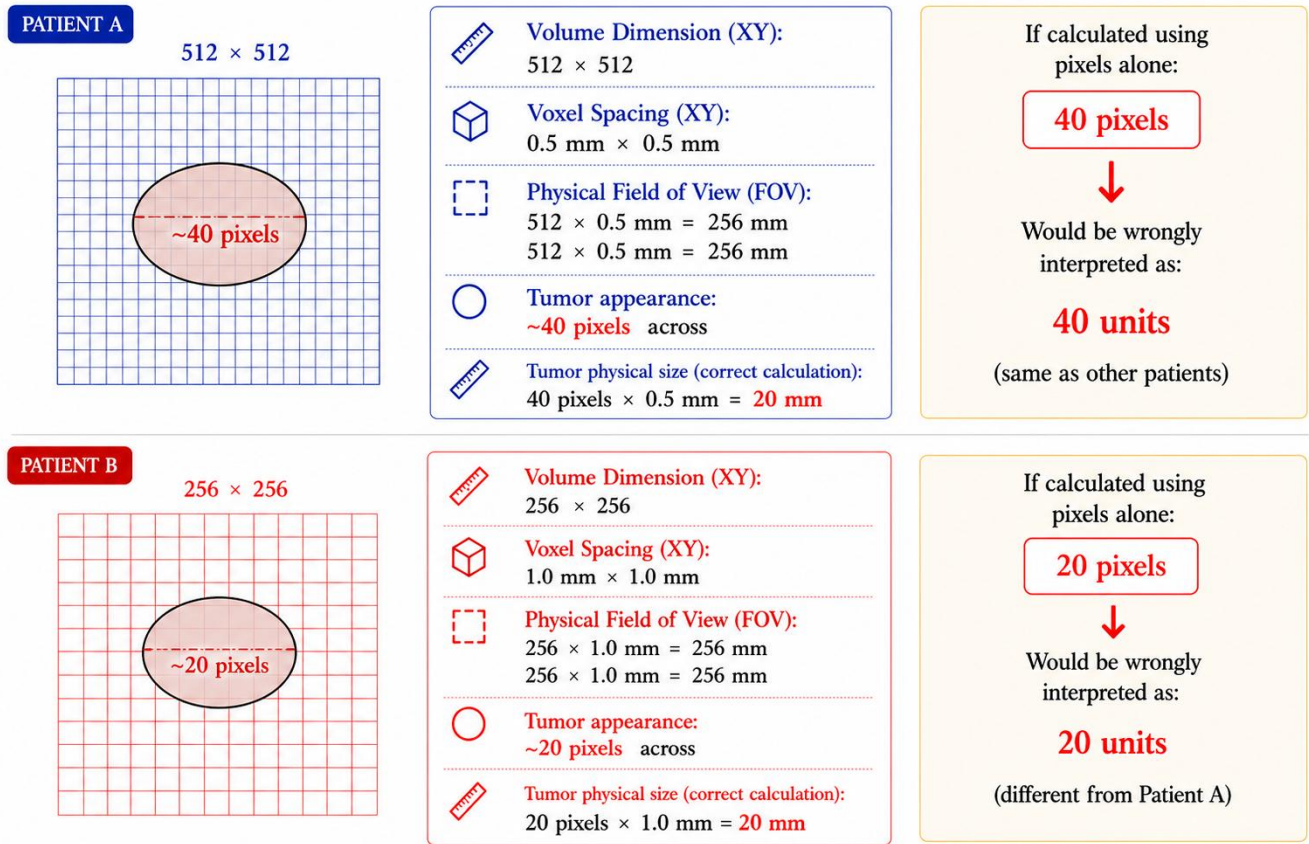


Figure 6 Effect of voxel spacing variation on anatomical pixel representation across patients. Although both tumors have the same physical size, differences in voxel spacing cause them to occupy different numbers of pixels, demonstrating the need for voxel standardization through resampling

As illustrated in Figure 6, resampling standardizes voxel spacing while preserving the physical anatomical dimensions of structures within the MRI volume. The implementation of voxel spacing standardization through resampling is provided in Appendix X under the file `2d_slices.py`, in the section titled “Resampling.”

2.2.4 Slice Extraction

Following resampling, each three-dimensional MRI volume was decomposed into a sequence of two-dimensional axial slices by extracting successive image planes along the slice (Z) direction. This process generated X–Y image slices corresponding to the axial anatomical plane while preserving the original slice ordering within each volume. To maintain accurate image–label correspondence, slice extraction was performed simultaneously for both the MRI volume and its associated segmentation mask, where each image slice (`img_data[:, :, z]`) was paired with the corresponding label

slice (`lbl_data[:, :, z]`) at the same slice index. Consequently, every extracted MRI slice retained its matching ground truth annotation, preserving spatial alignment and anatomical consistency throughout the dataset preparation process. This process converted each volumetric MRI dataset into a sequence of two-dimensional slices suitable for training using the implemented 2D U-Net architecture.

No additional slice removal or filtering was performed during preprocessing, and all available slices after resampling were retained for model preparation. The extracted image and label slices were subsequently stored as NumPy (`.npy`) arrays given in Appendix under the file `2d_slicing.py`, in the section titled “Save each slice as a NumPy array”, which preserve array structure and datatype while enabling efficient loading during deep learning model training.

2.2.5 Padding-Based Spatial Standardization to 400 X 400

A padding-based approach was implemented to generate uniform model input dimensions while preserving the anatomical representation established after voxel spacing standardization, no geometric scaling was applied to the extracted MRI slices or corresponding segmentation masks. Instead, slices with dimensions smaller than 400 X 400 pixels were standardized using symmetric zero padding. As a result, the number of pixels representing a given anatomical structure remained unchanged, while only the surrounding image boundaries were extended to achieve the required input dimensions.

Padding values for the height and width dimensions were computed separately based on the difference between the original slice dimensions and the target dimension of 400 X 400 pixels. Zero-valued borders were then added symmetrically around the image boundaries using NumPy’s `np.pad()` function, ensuring that anatomical structures remained centrally positioned within the image frame.

The same padding procedure was applied identically to both MRI intensity slices and segmentation masks to preserve spatial correspondence between image data and ground truth labels. This strategy was evaluated to investigate whether preserving the native anatomical representation of the resampled images improved segmentation performance compared with direct resizing approaches.

2.3 Model Training and Experimental Design

2.3.1 Dataset Loading and Organization

Following preprocessing, the generated MRI slices and corresponding segmentation masks stored in NumPy (*.npy*) format were loaded into the training pipeline. During the data preparation stage, the processed slices had already been organized into separate training and validation directories (*imagesTr*, *labelsTr*, *imagesVal*, and *labelsVal*) at the end of preprocessing after slicing. Therefore, the objective of this stage was not to create or organize the dataset again, but rather to retrieve the previously prepared files and supply them to the segmentation model.

File paths corresponding to the training and validation MRI slices and their associated segmentation masks were collected using Python's *glob.glob()* function. The *glob()* function searches the specified directories and retrieves the paths of all files matching the *.npy extension. The resulting file paths were subsequently arranged using Python's *sorted()* function to ensure that the ordering of image slices and segmentation masks remained consistent. At this stage, only the file path strings were obtained rather than loading the actual image data into memory. This strategy reduces memory consumption and improves computational efficiency, particularly when handling large medical imaging datasets containing thousands of image slices. The output of this step consisted of four ordered lists containing the file paths of training images, training labels, validation images, and validation labels.

The ordered image and label file path lists were subsequently paired using Python's *zip()* function, which generated tuples containing the corresponding MRI slice and segmentation mask file paths. These tuples were then organized into dictionaries using a list comprehension, where each dictionary contained the keys "image" and "label" storing the respective file paths. Each dictionary therefore represented a single MRI slice and its corresponding segmentation mask, while the actual image data remained on disk. This dictionary-based structure is required by MONAI because its dictionary transforms, including *LoadImaged()*, *EnsureChannelFirstd()*, *NormalizeIntensityd()*, *RandRotate90d()*, and *RandFlipd()*, operate by accessing the data through predefined dictionary keys. During preprocessing, *LoadImaged()* loads the corresponding .npy files into memory, while subsequent transforms automatically apply identical operations to both the image and its associated segmentation mask, thereby preserving spatial correspondence between the input image and the ground truth annotation. The final output of this stage

consisted of a list of training dictionaries and a list of validation dictionaries, with each list element containing the file paths of one MRI slice and its corresponding segmentation mask.

After organizing the file references, transformation pipelines were defined using MONAI's *Compose* function. The purpose of *Compose* is to combine multiple preprocessing and augmentation operations into a single sequential workflow. Rather than manually applying each operation independently, the transformations are stored as an ordered pipeline and executed automatically whenever a sample is requested. At this stage, no image processing was performed; only the transformation sequence was defined for later execution.

The training and validation dictionaries were subsequently supplied to MONAI's Dataset class. The Dataset object stores the image-label file references together with the transformation pipeline defined using *Compose*. At this stage, no images are loaded and no transformations are executed. Instead, the Dataset acts as a data source that can provide samples when requested by the DataLoader. During training and validation, the DataLoader retrieves samples from the Dataset, at which point the corresponding image and segmentation mask are loaded from disk and processed using the specified transformations. This design reduces memory consumption because transformed samples are generated only when required rather than being stored beforehand. The output of this step was a training dataset object and a validation dataset object containing the file references and transformation instructions required for subsequent data loading.

Finally, the dataset objects were supplied to MONAI's DataLoader class. The DataLoader is responsible for retrieving samples from the Dataset, applying the transformation pipeline, grouping samples into mini-batches, and supplying these batches to the neural network. A mini-batch refers to a small group of samples processed simultaneously during a single training iteration. In this study, a batch size of eight (`Batch_Size=8`) was used, meaning that eight MRI slices and their corresponding segmentation masks were processed together before updating the model parameters.

For the training dataset, shuffling was enabled (*`shuffle=True`*). This causes the DataLoader to randomly rearrange the order of samples before each training epoch, preventing the model from learning unintended patterns associated with the original dataset ordering and improving generalization performance. For the validation dataset, shuffling was disabled (*`shuffle=False`*) because validation is performed solely to evaluate model performance, and a fixed sample order ensures consistent and reproducible results.

Table 4-Comparison of data loading times obtained using different numbers of DataLoader worker processes

Num_Workers	Loading_Time_Sec
0	9.31
2	14.72
4	22.88
6	31.69
8	40.74

The `num_workers` parameter controls the number of CPU worker processes used by the `DataLoader` to load and preprocess data. To determine an appropriate configuration, a benchmarking study was performed using different `num_workers` values. The experiments were conducted on a workstation equipped with an NVIDIA T1000 GPU (4 GB), a 12-core CPU (20 logical processors), and 32 GB RAM.

As shown in Table 4, `num_workers=0` achieved the shortest loading time of 9.31 seconds for loading 50 batches of data, whereas larger values resulted in progressively longer loading times. Since the dataset consisted of pre-processed NumPy (`.npy`) files and only lightweight loading operations were required, the overhead associated with additional worker processes outweighed the benefits of parallel loading. Therefore, `num_workers=0` was selected for all subsequent experiments.

The `DataLoader` was configured with a batch size of 8, meaning that eight MRI slices and their corresponding segmentation masks were processed together in each mini-batch. The output of the `DataLoader` consisted of mini-batches of transformed MRI slices and segmentation masks in tensor format, which were supplied as input to the U-Net model.

2.3.2 Transformation Pipeline

After organizing the MRI slice and segmentation mask file paths into MONAI-compatible datasets, transformation pipelines were defined to automate data loading, preprocessing, augmentation, and tensor conversion. MONAI's `Compose` function was used to combine multiple transformations into a sequential workflow, ensuring that the same preprocessing operations were applied consistently whenever a sample was retrieved during training or validation.

Separate transformation pipelines were implemented for the training and validation datasets. Both pipelines included image loading, channel formatting, intensity normalization, and tensor

conversion. However, data augmentation operations were applied only to the training dataset to increase data variability and improve model generalization, while the validation dataset remained unmodified to enable unbiased performance evaluation as shown in Figure 7.

The first transformation, **LoadImaged**, loaded the preprocessed MRI slices and corresponding segmentation masks stored in NumPy (**.npy**) format. Subsequently **EnsureChannelFirstd** was

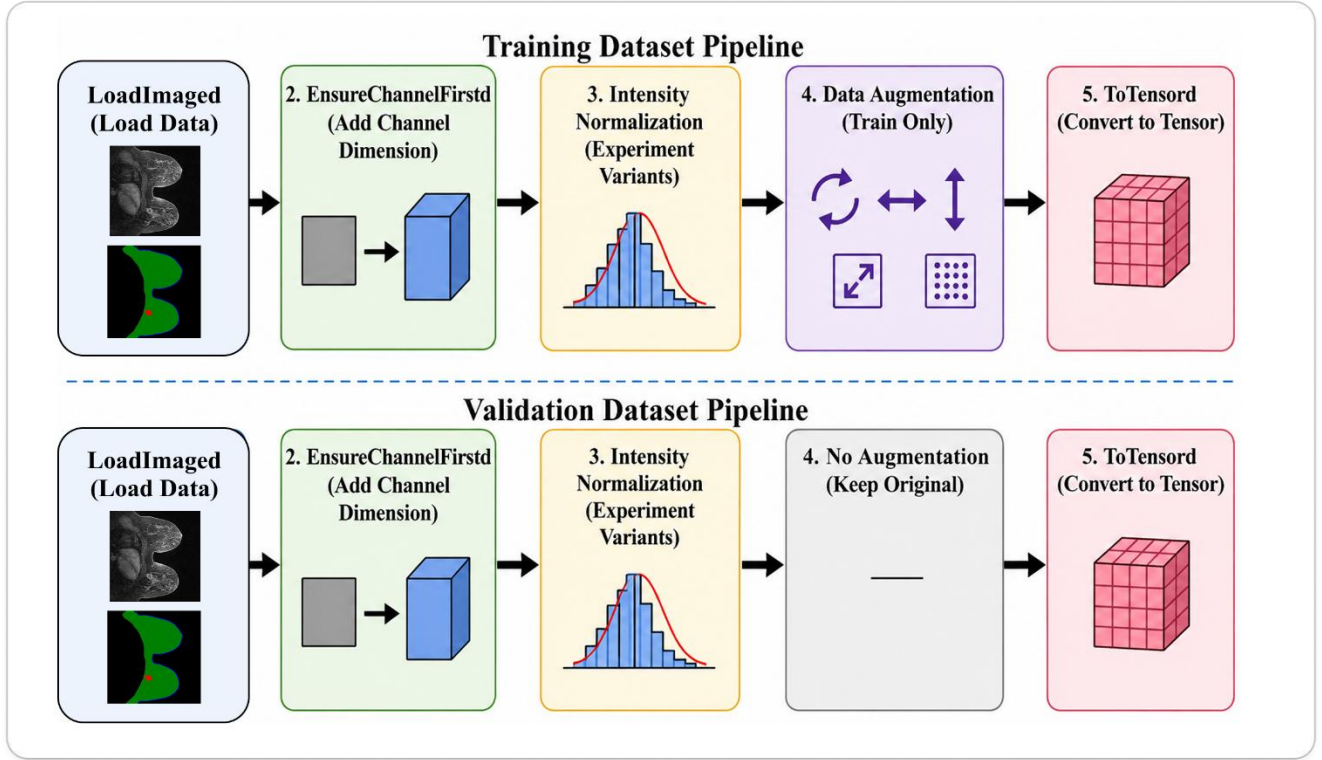


Figure 7 Overview of the training and validation transformation pipelines used for MRI slice preprocessing prior to U-Net training.

applied to explicitly convert the loaded MRI slices and segmentation masks into channel-first format (C, H, W), which is the input format expected by MONAI and PyTorch convolutional neural networks. Since the MRI slices are grayscale images, they originally contain only height and width dimensions (H, W).

EnsureChannelFirstd adds an explicit channel dimension, converting the data to (1, H, W), where **1** represents a single image intensity channel. This transformation does not create new image information or alter the pixel values; rather, it wraps the existing image data within a channel container so that the image is explicitly represented as a one-channel input. By organizing the data in channel-first

format (C, H, W), the transform enables MONAI and PyTorch to correctly interpret the MRI slices as single-channel grayscale images and ensures compatibility with the U-Net architecture configured with `in_channels = 1`.

Following channel formatting, intensity normalization was applied to reduce inter-scan intensity variability and improve the consistency of MRI input data. Rather than removing diagnostically relevant high-intensity regions, normalization preserves the relative contrast between anatomical structures while mapping the intensity values to a standardized range. This enables the segmentation model to learn tissue characteristics that are less dependent on scanner-specific intensity scales. As part of the experimental study, two different normalization methods were evaluated independently to assess their effect on segmentation performance. The methodology and implementation of each normalization approach are discussed in the subsequent sections.

2.3.3 Data Augmentation

Random 90-degree rotations and horizontal and vertical flipping operations were applied to increase spatial variability within the training dataset and improve model generalization. Although such transformations may not naturally occur in clinical MRI acquisitions, they help prevent the segmentation network from memorizing specific anatomical positions and orientations present in the training data.

Instead, the model is encouraged to learn anatomy-specific features based on tissue appearance, shape, and surrounding context. All augmentation operations as shown in Figure 8 applied simultaneously to each MRI image and its corresponding segmentation mask within the training dataset to preserve image-label alignment, while the validation dataset remained unmodified to enable unbiased performance evaluation.

Random rotations were implemented using MONAI's `RandRotate90d` transform, which rotates MRI slices and their corresponding segmentation masks by multiples of 90° within the image plane. The parameter `prob = 0.5` specifies that the transformation is applied with a probability of 50% whenever a training sample is loaded. Horizontal (`spatial_axis=1`) and vertical flipping (`spatial_axis=0`) operations were implemented using MONAI's `RandFlipd` transform, which reflect the image across the vertical and horizontal axes, respectively. Each flip was applied independently with a probability of 50%.

Furthermore, augmentation was performed online during training. This was achieved through MONAI's random transformation functions, specifically `RandFlip()` and `RandRotate90d()`, which are executed each time the `DataLoader` retrieves a sample from the Dataset. Consequently, transformed MRI slices and segmentation masks were generated dynamically during data loading rather than being permanently stored on disk. Consequently, the number of stored training images remained unchanged, while the effective diversity of samples observed by the network increased substantially, reducing overfitting and improving robustness to unseen data.

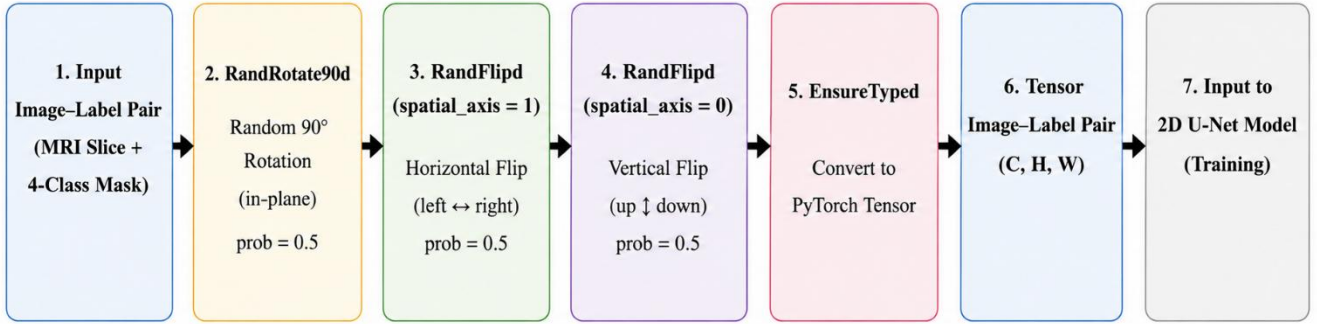


Figure 8 Training dataset augmentation workflow implemented using MONAI transforms. Random rotations and flipping operations were applied online to both MRI slices and corresponding segmentation masks before conversion to PyTorch tensor format and input to the 2D U-Net Model

Finally, `EnsureTyped()` converted the processed MRI slices and segmentation masks from NumPy arrays into PyTorch tensor representations for both the training and validation datasets. The image-label pairing was preserved by maintaining the data in dictionary format with the predefined "image" and "label" keys. During batch formation, the `DataLoader` combined these dictionaries into mini-batches of image and label tensors, where the image tensors were supplied as input to the 2D U-Net model and the corresponding label tensors were used as ground truth for training. The output of the transformation pipeline therefore consisted of normalized, properly formatted, spatially aligned, and tensor-based image-label pairs ready for model training and validation.

2.3.4 Intensity Normalization

Intensity normalization was investigated as part of the experimental design to evaluate its influence on segmentation performance. MRI images do not possess a standardized intensity scale, and intensity values may vary substantially across scanners, acquisition protocols, and patients. Such

variations can affect training stability and may reduce the ability of the segmentation network to learn consistent anatomical features. To investigate the impact of two different normalization strategies, five normalization configurations were evaluated as below.

Experiment E1: No Intensity Normalization

In the first experiment, no intensity normalization was applied to the MRI slices. The images and corresponding segmentation masks were loaded using MONAI's `LoadImaged()` transform, converted into channel-first format using `EnsureChannelFirstd()`, and subsequently transformed into PyTorch tensor representations using `EnsureTyped()`. The original MRI intensity values were preserved throughout the training process without any modification. This experiment served as the baseline configuration against which all normalization-based approaches were compared.

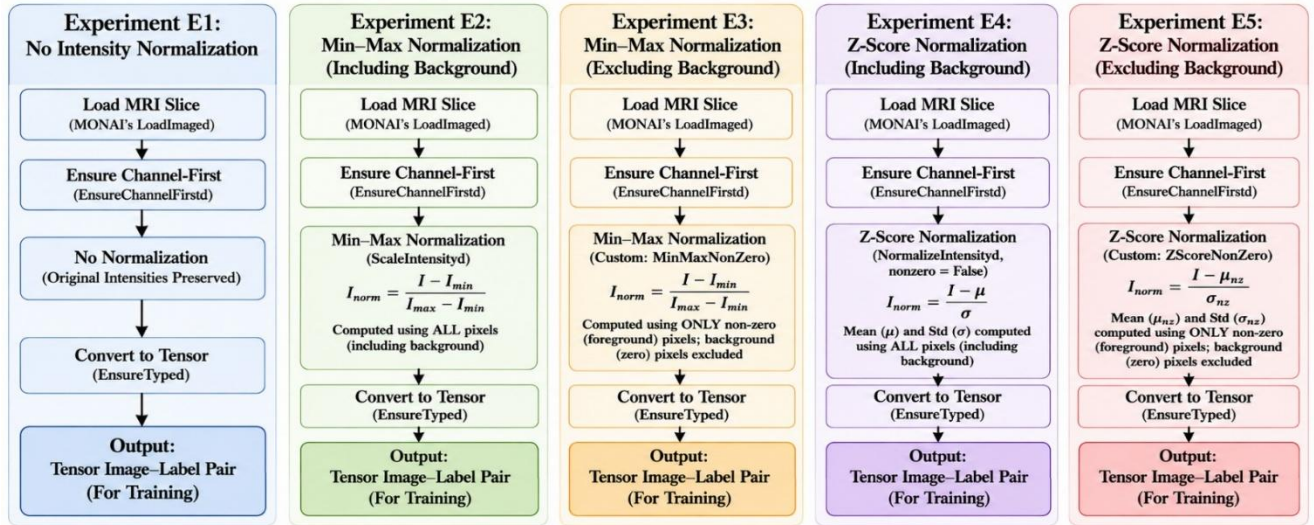


Figure 9 Experimental intensity normalization configurations evaluated in this study, including no normalization, Min-Max normalization (with and without background), and Z-score normalization (with background) applied during the MRI preprocessing pipeline

Experiment E2: Min-Max Normalization

In the second experiment, intensity normalization was performed using MONAI's `ScaleIntensityd` transform. This operation linearly scaled the image intensities to the range [0, 1] according to

$$I_{norm} = \frac{I - I_{min}}{I_{max} - I_{min}} \quad (ii)$$

where I represents the original pixel intensity, I_{min} represents the minimum intensity value within the image, and I_{max} represents the maximum intensity value. This approach preserves the relative intensity differences between tissues while scaling all intensity values to a common range of 0 to 1. For example, if a tumor has an intensity value of 500 and surrounding tissue has an intensity value of 250 before normalization, the tumor will still appear relatively brighter than the surrounding tissue after normalization. By converting all MRI scans to the same numerical range, intensity variations between patients are reduced, resulting in more stable and consistent model training. For example, the same tumor may appear with an intensity value of 500 in one scan and 1500 in another due to differences in image acquisition. Intensity normalization reduces these variations by transforming the images to a common scale, thereby improving consistency during model training. Since *ScaleIntensityd* performs normalization using all image pixels, the minimum and maximum intensity values were computed from the entire image, including zero-valued background regions. Consequently, background pixels contributed to the normalization process.

Experiment E3: Min–Max Normalization Excluding Background

In the third experiment, a custom MONAI MapTransform class named **MinMaxNonZero** was implemented to perform Min–Max normalization while excluding background pixels from the normalization statistics. The custom transform was created by inheriting from MONAI's MapTransform class and was applied to the MRI images during the transformation pipeline. For each image, a binary mask (True/False) was first generated to identify non-zero voxels corresponding to anatomically relevant regions. The minimum and maximum intensity values were then computed exclusively from these non-zero pixels, while zero-valued background pixels were excluded from the calculation. Subsequently, the identified tissue intensities were linearly scaled to the range [0,1] according to equation (ii), where I represents the original voxel intensity, and I_{min} and I_{max} represent the minimum and maximum intensity values computed only from the non-background region. Background pixels remained unchanged throughout the normalization process. By excluding background voxels from the normalization statistics,

the full normalization range was utilized by anatomically relevant tissues rather than being influenced by the large number of zero-valued background pixels commonly present in MRI images.

Experiment E4: Z-Score Normalization Including Background

In the fourth experiment, intensity standardization was performed using MONAI's *NormalizeIntensity* transform with *nonzero=False* thereby including background pixels in the normalization process. The mean (μ) and standard deviation (σ) were calculated using all image pixels, including background regions. The image intensities were subsequently standardized according to:

$$I_{norm} = \frac{I - \mu}{\sigma} \quad iii$$

This transformation produces intensity distributions with approximately zero mean and unit variance, thereby reducing intensity scale differences between MRI scans. Since Z-score normalization is a linear transformation, it preserves the relative intensity differences between neighboring pixels. Specifically, the difference between two adjacent pixel intensities after normalization is equal to the original intensity difference divided by the image standard deviation. Consequently, local contrast and smooth anatomical transitions are maintained, while only the overall intensity scale of the MRI scan is standardized.

2.3.5 Optimization Strategy and Training Procedure

The proposed 2D U-Net model was trained using the PyTorch and MONAI frameworks. Following preprocessing, the training and validation datasets were organized into image-label pairs and loaded using PyTorch *DataLoader* objects, which automatically generated mini-batches for efficient GPU processing. A batch size of eight was selected to provide a balance between GPU memory utilization and stable gradient estimation, while training was performed for a maximum of 50 epochs. The model was executed on an NVIDIA GPU when available through the

`torch.device("cuda:0")` setting which is to use first GPU. The system used for this work had only one GPU.

The implemented U-Net was configured for two-dimensional image segmentation (`spatial_dims = 2`) with a single-channel grayscale MRI `input (in_channels = 1)` and four output classes (`out_channels = 4`), corresponding to background, tumor, glandular tissue, and skin respectively. The encoder-decoder architecture employed feature channel sizes of `(16, 32, 64, 128, 256)`, allowing the network to progressively learn increasingly complex anatomical representations. Spatial downsampling was performed using stride values of `(2, 2, 2, 2)`, reducing the feature map dimensions by a factor of two at each encoder stage. Additionally, two residual convolutional units (`num_res_units = 2`) were incorporated at each level to facilitate stable gradient propagation and improved feature learning.

Model optimization was performed using the Adam (Adaptive Moment Estimation) optimizer implemented in PyTorch. Adam was selected because it combines momentum-based optimization with adaptive learning rate adjustment, enabling efficient and stable convergence during deep learning training. The optimizer was initialized using the trainable parameters of the U-Net through the `model.parameters()` function, which provides access to all learnable weights and biases within the network. A learning rate of 1×10^{-4} was employed throughout training to control the magnitude of parameter updates.

For each mini-batch, MRI slices were passed through the U-Net using a forward propagation step to generate predicted segmentation maps. The discrepancy between the predicted outputs and the corresponding ground truth masks was computed using the MONAI `DiceCELoss` function, which combines Dice Loss and Cross-Entropy Loss. In the present implementation, both components contributed equally to the total loss (`lambda_dice = 0.5, lambda_ce = 0.5`). The `softmax=True` option converted the network outputs into class probabilities, while `to_onehot_y=True` automatically transformed the ground truth labels into one-hot encoded representations required for multi-class loss computation.

After loss calculation, the `loss.backward()` function performed backpropagation to compute gradients for all trainable network parameters. Subsequently, `optimizer.step()` updated

the model weights according to the Adam optimization algorithm, while `optimizer.zero_grad()` cleared previously accumulated gradients before processing the next mini-batch.

At the end of each epoch, the model was evaluated using the validation dataset. During validation, the network was switched to evaluation mode using `model.eval()`, and gradient computation was disabled through the `torch.no_grad()` context to reduce computational overhead and ensure that network parameters remained unchanged. The final class prediction for each pixel was obtained using the `torch.argmax()` function, which selected the class with the highest predicted probability among the four output channels.

Segmentation performance on the validation dataset was quantified using MONAI's `DiceMetric` with `include_background=True`, thereby evaluating along with background. The mean Dice score across all validation samples was recorded after each epoch. Whenever a new highest validation Dice score was achieved, the corresponding model parameters were preserved using `torch.save()` as the best-performing checkpoint for subsequent inference and performance evaluation. Through repeated execution of forward propagation, loss computation, backpropagation, parameter updates, and validation, the network progressively learned increasingly accurate multi-class segmentation maps for breast MRI images.

2.4 Inference Procedure

Following completion of training, inference was performed on the 10 previously unseen breast MRI volumes contained in the testing dataset. For each experiment, the best-performing model checkpoint, selected based on the highest validation Dice score, was loaded and switched to evaluation mode using PyTorch's `model.eval()` function. In addition, inference was executed within a `torch.no_grad()` context to disable gradient computation, thereby reducing memory consumption and improving computational efficiency.

The test MRI volumes were loaded using the NiBabel library and underwent the same preprocessing pipeline applied during training, including voxel-spacing standardization, slice extraction, padding operations, and the corresponding intensity normalization strategy. Maintaining identical preprocessing during training and inference ensured consistency in the spatial and intensity characteristics of the input data.

Each preprocessed slice was converted into a PyTorch tensor and provided as input to the trained MONAI 2D U-Net model. Figure 10 illustrates the inference process, where the trained network generates four class-specific output feature maps and the final segmentation mask is obtained by assigning each pixel to the class with the highest predicted score using the `argmax()` operation.

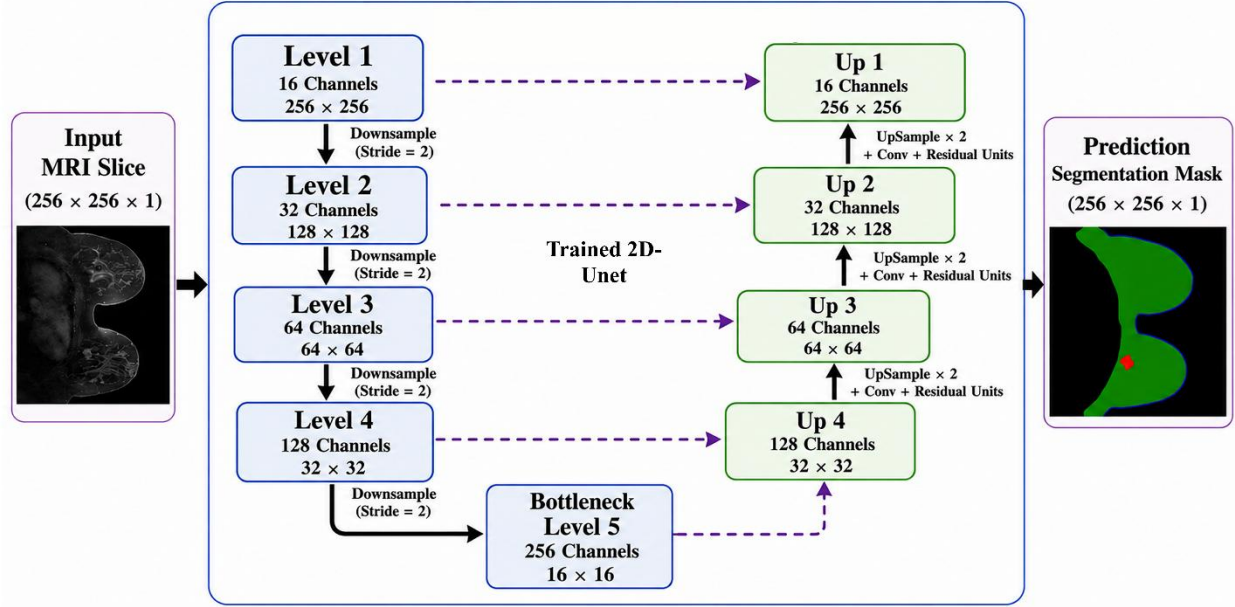


Figure 10 Inference workflow of the trained 2D U-Net model. The input MRI slice is processed through the trained network to produce a pixel-wise multi-class segmentation mask

Following prediction, any preprocessing transformations such as padding were reversed, and the individual slice predictions were reconstructed into a complete three-dimensional segmentation volume. The final segmentation was saved in NIfTI format using the original affine transformation matrix to preserve anatomical orientation and spatial correspondence with the source MRI volume.

The generated segmentation volumes were subsequently compared with the corresponding ground truth annotations using quantitative evaluation metrics including Dice Similarity Coefficient (DSC), Intersection over Union (IoU), Precision, and Recall. Finally, post-processing operations were applied to reduce false-positive detections and improve anatomical consistency prior to final analysis.

2.5 Post-Processing

Following inference, post-processing was applied to the predicted segmentation volumes to reduce false-positive tumor detections and improve the anatomical plausibility of the final segmentation results. The post-processing pipeline consisted of two sequential stages: (i) removal of small tumor components

based on physical volume compared with smallest tumor size in all the 40-dataset including test and train dataset and (ii) removal of spatially isolated tumor components located far from the primary tumor region.

2.5.1 Removal of Small Tumor Components

Following segmentation, a post-processing step was applied to reduce false-positive tumor detections. Initially, the ground truth annotation volumes of all 40 training cases were analyzed to determine the distribution of tumor volumes. For each case, the total tumor volume was calculated by counting the number of tumor voxels and multiplying by the corresponding voxel dimensions obtained from the NIfTI image metadata. The calculated tumor volumes were then arranged in descending order to visualize the volume distribution across the dataset as shown in Figure 11 .

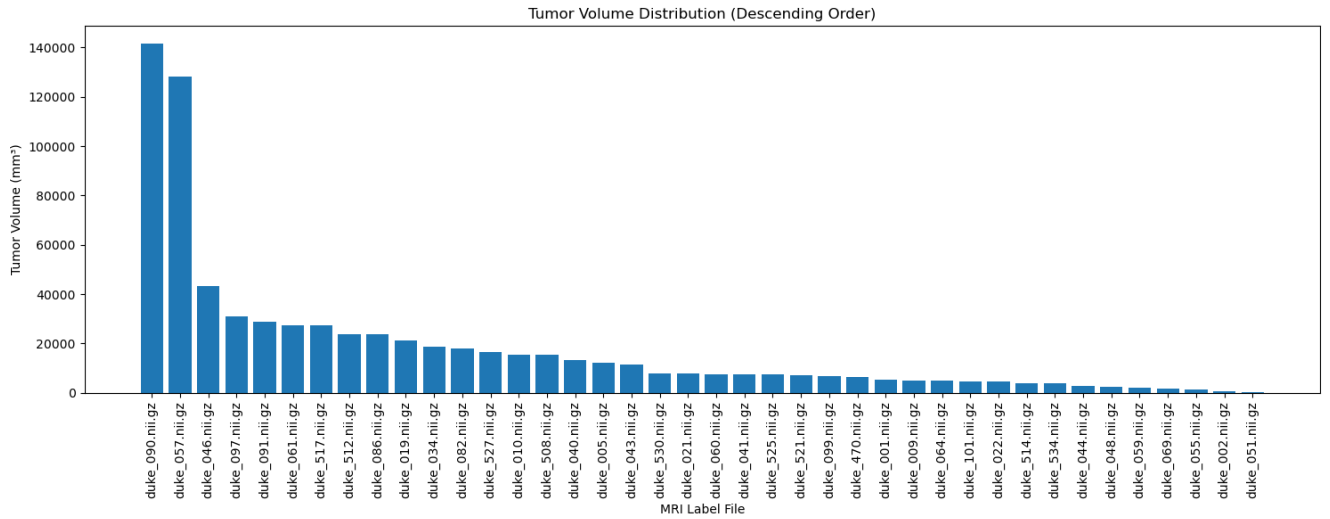


Figure 11 Graphical representation of the distribution of ground-truth tumor volumes across the 40 annotated MRI cases arranged in descending order.

Based on this analysis, a minimum tumor volume threshold of **377.89 mm³** was selected. During post-processing, each predicted segmentation volume was examined to identify individual tumor regions labeled as the tumor class. The physical volume of each detected tumor region was calculated, and regions having a volume smaller than the predefined threshold were removed by reassigning them to the background class. Tumor regions with volumes greater than or equal to the threshold were preserved.

The resulting cleaned segmentation volumes were subsequently saved in NIfTI format while preserving the original image header information and affine transformation matrix to maintain anatomical orientation and spatial correspondence with the source MRI volumes.

2.5.2 Removal of Spatially Isolated Tumor Components

Although the initial post-processing step successfully removed very small tumor predictions, some relatively large false-positive regions still remained. These regions were typically located far away from the primary tumor mass and were anatomically unlikely to represent true tumor tissue. Therefore, a second post-processing step was introduced to remove such distant false-positive detections.

After applying the minimum volume threshold, all remaining tumor regions within each predicted segmentation volume were identified. The tumor region with the largest physical volume was designated as the main tumor region, while all other remaining tumor regions were treated as secondary tumor regions. For every secondary region, the geometric center (centroid) was calculated and its Euclidean distance from the centroid of the main tumor region was determined in physical units (millimeters).

To establish an objective distance threshold, the distances obtained from all test cases were collected and their median value was calculated. The median was selected instead of the mean because it is less sensitive to extremely large or small distances and therefore provides a more robust estimate of the typical spatial separation between anatomically related tumor regions.

During post-processing, any secondary tumor region whose centroid distance exceeded the calculated median threshold was considered an isolated false-positive detection and was removed by reassigning its voxels to the background class. Secondary tumor regions located within the threshold distance were preserved, since they were more likely to represent anatomically connected or clinically plausible tumor extensions.

A median centroid-distance threshold of **63.69 mm** was determined from the dataset and used to eliminate anatomically distant false-positive tumor predictions while preserving plausible neighboring tumor regions. Table 5 presents representative examples of the calculated tumor volumes, centroid distances, and the corresponding keep/remove decisions, also there is one tumor out false which was not able to remove is marked as “couldn’t remove” in the table. As shown, several secondary tumor regions

Table 5 Analysis of tumor region volumes and centroid distances used for distance-based post-processing.

File	Total Components	Role	Component	Volume (mm ³)	Distance from Main (mm)	Keep?
duke_470_0001.nii	2	Main	1	5610.23	0	Keep
duke_470_0001.nii	2	Secondary	2	862.94	84.92	Remove
duke_508_0001.nii	3	Secondary	1	797.88	80.76	Remove
duke_508_0001.nii	3	Secondary	2	434.04	30.78	Keep
duke_508_0001.nii	3	Main	3	10235.04	0	Keep
duke_512_0001.nii	2	Secondary	1	596.92	128.47	Remove
duke_512_0001.nii	2	Main	2	791.63	0	Keep
duke_514_0001.nii	1	Main	1	6112.14	0	Keep
duke_517_0001.nii	3	Main	1	7698.45	0	Keep
duke_517_0001.nii	3	Secondary	2	2855.26	26.41	Keep
duke_517_0001.nii	3	Secondary	3	581.8	30.56	Keep
duke_521_0001.nii	1	Main	1	3564.84	0	Keep
duke_525_0001.nii	2	Main	1	1751.73	0	Keep
duke_525_0001.nii	2	Secondary	2	399.37	149.33	Remove
duke_527_0001.nii	7	Main	1	34487.63	0	Keep
duke_527_0001.nii	7	Secondary	2	409.39	61.61	Keep
duke_527_0001.nii	7	Secondary	3	717.4	63.69	Keep
duke_527_0001.nii	7	Secondary	4	932.43	57.87	Keep
duke_527_0001.nii	7	Secondary	5	1168.12	43.95	Keep
duke_527_0001.nii	7	Secondary	6	5637.2	193.2	Remove
duke_527_0001.nii	7	Secondary	7	419.72	206.24	Remove
duke_530_0001.nii	1	Main	1	1133	0	Keep
duke_534_0001.nii	3	Main	1	1496.8	0	Keep
duke_534_0001.nii	3	Secondary	2	617.31	83.22	Couldn't remove
duke_534_0001.nii	3	Secondary	3	794.89	24.45	Keep

located at large distances from the primary tumor were successfully eliminated, whereas nearby secondary regions were retained. This distance-based filtering substantially reduced false-positive tumor detections while preserving the primary tumor morphology and anatomically consistent neighboring tumor regions.

2.6 Results and Discussion

2.6.1 Overview of Experiments

To investigate the influence of different preprocessing and post-processing strategies on breast MRI segmentation performance, multiple experiments were conducted using the proposed 2D U-Net MONAI framework. Each experiment was evaluated quantitatively using Dice Similarity Coefficient (DSC), Intersection over Union (IoU), Precision, and Recall, and qualitatively through visual comparison of the predicted segmentation masks with expert segmentation.

2.6.2 Evaluation of Spatial Standardization through Resampling and Padding

The proposed preprocessing pipeline consisted of two sequential spatial standardization steps: voxel spacing normalization followed by padding-based dimension standardization. Initially, all MRI volumes were resampled to a common in-plane spatial resolution while preserving the original slice thickness, ensuring consistent physical representation of anatomical structures across patients. Since variations in image dimensions still remained after resampling due to differences in acquisition protocols, symmetric zero-padding was subsequently applied after slice extraction to generate fixed-size inputs compatible with the 2D U-Net architecture.

Table 6 summarizes the effect of the resampling operation on the 30 MRI volumes included in the training dataset. The original in-plane voxel spacing varied from approximately 0.5859 mm to 1.0625 mm because of differences in imaging protocols. Following preprocessing, all images were standardized to a common spatial resolution while maintaining the original z-direction slice spacing.

Table 6 summarizes the effect of the resampling operation on the 30 MRI volumes included in the training dataset. The original in-plane voxel spacing varied from approximately 0.5859 mm to 1.0625 mm because of differences in imaging protocols. Following

Sample ID	Original Volume Dimension (In-plane)	Original Voxel Spacing (mm)	Resampled Volume Dimension (In-plane)	Resampled Voxel Spacing (mm)
duke_001_0000.nii.gz	(448, 448)	(0.8036, 0.8036)	(360, 360)	(1.0000, 1.0000)
duke_002_0000.nii.gz	(512, 512)	(0.5859, 0.5859)	(300, 300)	(1.0000, 1.0000)

duke_005_0000.nii.gz	(448, 448)	(0.7812, 0.7812)	(350, 350)	(1.0000, 1.0000)
duke_009_0000.nii.gz	(512, 512)	(0.6641, 0.6641)	(340, 340)	(1.0000, 1.0000)
duke_010_0000.nii.gz	(512, 512)	(0.6641, 0.6641)	(340, 340)	(1.0000, 1.0000)
duke_019_0000.nii.gz	(320, 320)	(1.0625, 1.0625)	(340, 340)	(1.0000, 1.0000)
duke_021_0000.nii.gz	(512, 512)	(0.7422, 0.7422)	(380, 380)	(1.0000, 1.0000)
duke_022_0000.nii.gz	(448, 448)	(0.7366, 0.7366)	(330, 330)	(1.0000, 1.0000)
duke_034_0001.nii.gz	(512, 512)	(0.7031, 0.7031)	(360, 360)	(1.0000, 1.0000)
duke_040_0001.nii.gz	(448, 448)	(0.7589, 0.7589)	(340, 340)	(1.0000, 1.0000)
duke_041_0001.nii.gz	(512, 512)	(0.6836, 0.6836)	(350, 350)	(1.0000, 1.0000)
duke_043_0001.nii.gz	(512, 512)	(0.6836, 0.6836)	(350, 350)	(1.0000, 1.0000)
duke_044_0001.nii.gz	(448, 448)	(0.7589, 0.7589)	(340, 340)	(1.0000, 1.0000)
duke_046_0001.nii.gz	(448, 448)	(0.8371, 0.8371)	(375, 375)	(1.0000, 1.0000)
duke_048_0001.nii.gz	(448, 448,)	(0.7589, 0.7589)	(340, 340)	(1.0000, 1.0000)
duke_051_0001.nii.gz	(512, 512)	(0.7422, 0.7422)	(380, 380)	(1.0000, 1.0000)
duke_055_0001.nii.gz	(512, 512)	(0.6445, 0.6445)	(330, 330)	(1.0000, 1.0000)
duke_057_0001.nii.gz	(448, 448)	(0.8036, 0.8036)	(360, 360)	(1.0000, 1.0000)
duke_059_0001.nii.gz	(512, 512)	(0.6641, 0.6641)	(340, 340)	(1.0000, 1.0000)
duke_060_0001.nii.gz	(512, 512)	(0.6836, 0.6836)	(350, 350)	(1.0000, 1.0000)
duke_061_0001.nii.gz	(512, 512)	(0.6641, 0.6641)	(340, 340)	(1.0000, 1.0000)
duke_064_0001.nii.gz	(512, 512)	(0.6445, 0.6445)	(330, 330)	(1.0000, 1.0000)
duke_069_0001.nii.gz	(512, 512)	(0.6641, 0.6641)	(340, 340)	(1.0000, 1.0000)
duke_082_0001.nii.gz	(448, 448)	(0.7589, 0.7589)	(340, 340)	(1.0000, 1.0000)
duke_086_0001.nii.gz	(448, 448)	(0.7812, 0.7812)	(350, 350)	(1.0000, 1.0000)
duke_090_0001.nii.gz	(512, 512)	(0.6836, 0.6836)	(350, 350)	(1.0000, 1.0000)

duke_091_0001.nii.gz	(448, 448)	(0.7589, 0.7589)	(340, 340)	(1.0000, 1.0000)
duke_097_0001.nii.gz	(448, 448)	(0.7812, 0.7812)	(350, 350)	(1.0000, 1.0000)
duke_099_0001.nii.gz	(512, 512)	(0.6250, 0.6250)	(320, 320)	(1.0000, 1.0000)
duke_101_0001.nii.gz	(448, 448)	(0.6920, 0.6920)	(310, 310)	(1.0000, 1.0000)

As expected, modifying the voxel spacing resulted in corresponding changes to the image dimensions. Images with smaller original voxel spacing (for example, 0.5859 mm) experienced a reduction in pixel dimensions after resampling, whereas images with larger original spacing (such as 1.0625 mm) showed an increase in image size. Consequently, the resampled in-plane dimensions ranged from 300×300 pixels to 380×380 pixels.

Although voxel spacing standardization ensured anatomical consistency across the dataset, the variation in image dimensions remained. Since the proposed 2D U-Net architecture requires a fixed input size, an additional padding step was introduced after slice extraction. Symmetric zero-padding was applied around the image boundaries to produce uniform input dimensions without altering the anatomical structures or tumor morphology.

Figure 12 illustrates the effect of voxel spacing standardization through resampling on representative breast MRI slices. The top row shows the original images acquired with different in-plane voxel spacings, while the bottom row shows the corresponding images after resampling to a uniform voxel spacing of 1.0×1.0 mm. It should be noted that resampling does not physically shrink or enlarge the breast anatomy. Instead, the procedure redistributes the image information onto a new voxel grid, causing the number of pixels required to represent the same anatomical structure to change. Consequently, the differences observed along the x- and y-axis represent changes in image dimensions (pixel count) rather than changes in the actual physical size of the breast or tumor. The underlying anatomical structures and morphology remain unchanged, while only the spatial sampling resolution is standardized across patients.

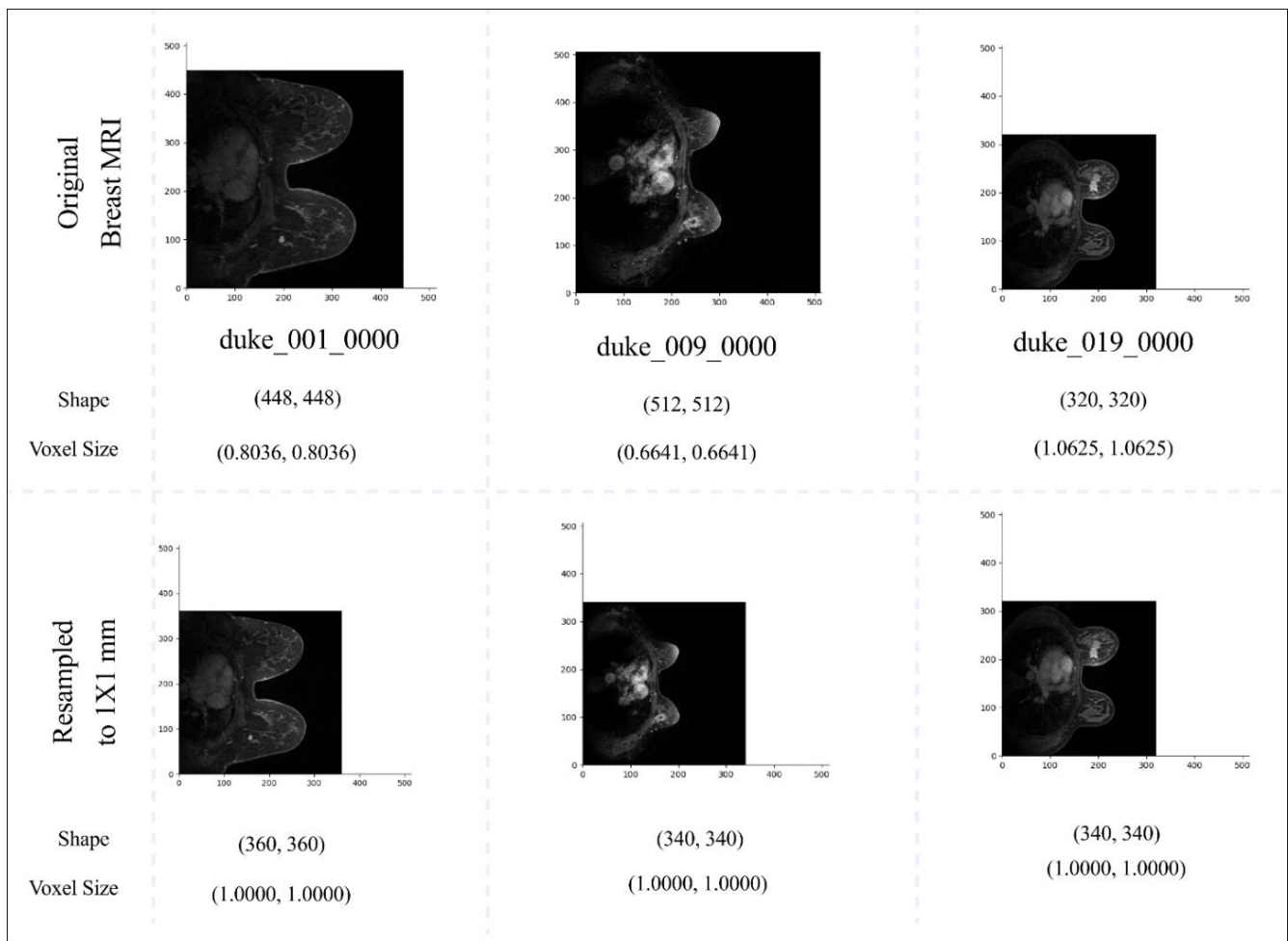


Figure 12 Visual comparison of representative breast MRI slices before and after voxel spacing standardization through resampling. The observed changes correspond to image dimensions (pixel representation) resulting from voxel redistribution and do not indicate any physical change in breast size or tumor morphology.

Figure 13 Representative breast MRI slices before and after padding to 400×400 pixels. In the Figure 13 top row shows representative resampled breast MRI slices with varying image dimensions (360×360 and 340×340 pixels), reflecting differences that remain after resampling. The bottom row presents the corresponding images after symmetric zero-padding to a fixed size of 400×400 pixels.

Table 7 Original and resampled in-plane image sizes and voxel spacings of the test MRI dataset.

Sample ID	Original Image Size (H x W)	Original In-plane Spacing (mm)	Resampled Image Size (H x W)	Resampled In-plane Spacing (mm)
duke_470_0001.nii	(512, 512)	(0.7227, 0.7227)	(370, 370)	(1.0000, 1.0000)
duke_508_0001.nii	(512, 512)	(0.7031, 0.7031)	(360, 360)	(1.0000, 1.0000)
duke_512_0001.nii	(448, 448)	(0.7812, 0.7812)	(350, 350)	(1.0000, 1.0000)
duke_514_0001.nii	(512, 512)	(0.7031, 0.7031)	(360, 360)	(1.0000, 1.0000)
duke_517_0001.nii	(512, 512)	(0.6836, 0.6836)	(350, 350)	(1.0000, 1.0000)
duke_521_0001.nii	(512, 512)	(0.6250, 0.6250)	(320, 320)	(1.0000, 1.0000)
duke_525_0001.nii	(512, 512)	(0.7422, 0.7422)	(380, 380)	(1.0000, 1.0000)
duke_527_0001.nii	(448, 448)	(0.8036, 0.8036)	(360, 360)	(1.0000, 1.0000)
duke_530_0001.nii	(512, 512)	(0.6641, 0.6641)	(340, 340)	(1.0000, 1.0000)
duke_534_0001.nii	(448, 448)	(0.8036, 0.8036)	(360, 360)	(1.0000, 1.0000)

Padding increased the image dimensions by enlarging the image canvas through the addition of zero-valued background pixels around the image boundaries, while preserving the standardized voxel spacing and maintaining the original anatomical structures and lesion morphology. Unlike resizing operations, padding does not modify pixel intensities or alter the physical scale of the underlying anatomy; instead, it simply introduces additional background regions surrounding the original image content. Symmetric padding was employed to keep the breast anatomy approximately centered within the image, thereby avoiding any systematic positional bias toward a particular edge of the input. Providing images with identical spatial dimensions was essential because the 2D U-Net architecture processes mini-batches of images that must share the same input size. Consequently, the padding operation enabled efficient and consistent batch processing while preserving the anatomical integrity and spatial representation established during the preceding resampling step. The padding is applied as below Figure 13.

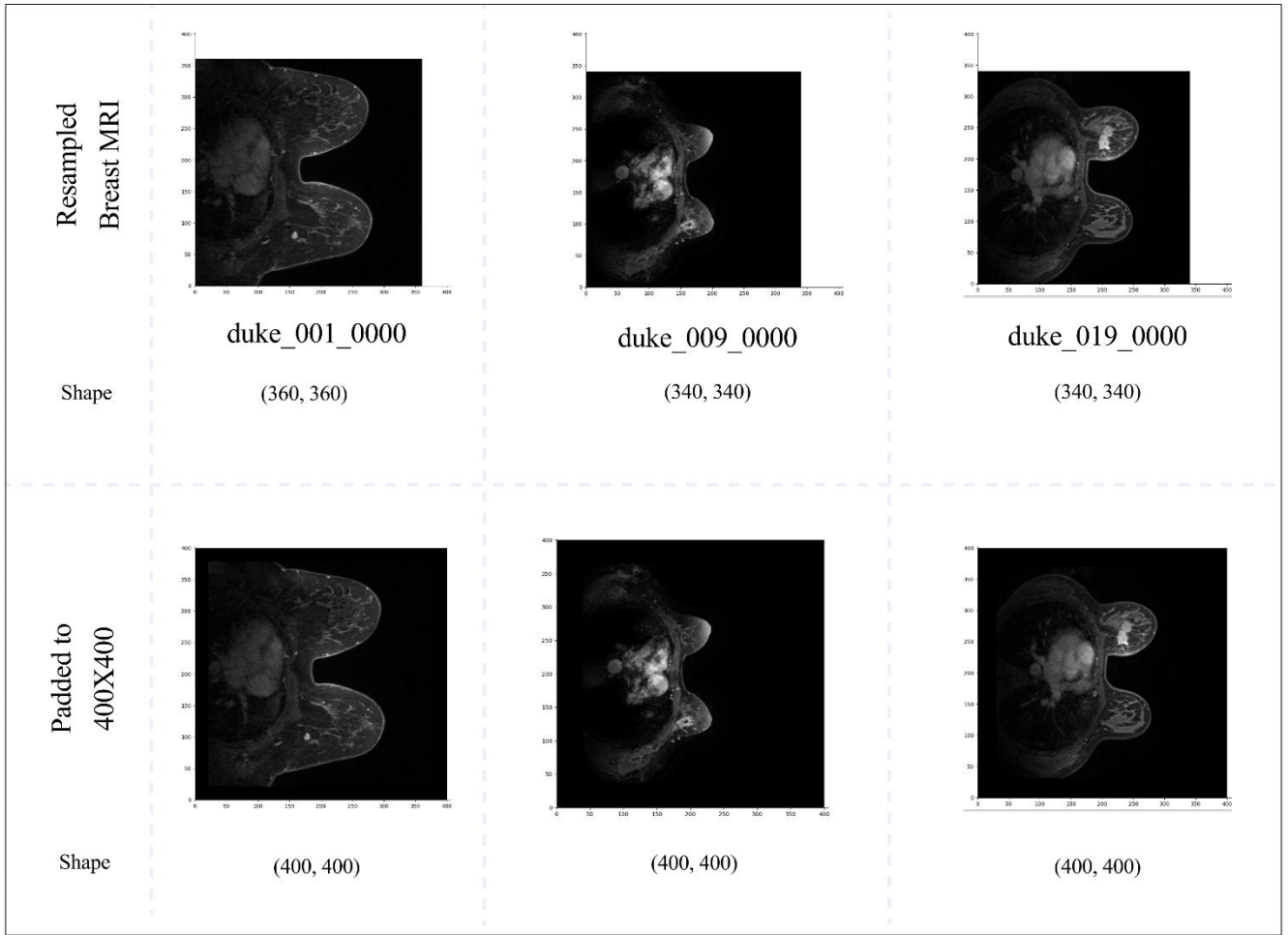


Figure 13 Representative breast MRI slices before and after padding to 400×400 pixels.

2.6.3 Evaluation of Intensity Normalization Methods

To investigate the influence of intensity normalization on segmentation performance, four preprocessing strategies were evaluated: no normalization, Min-Max normalization, Min-Max normalization excluding background pixels, and Z-score normalization. Model performance was assessed using the best validation Dice score obtained during training and the independent test-set metrics, namely Dice Similarity Coefficient (DSC), Intersection over Union (IoU), Precision, and Recall. These metrics are defined as

$$Dice = \frac{2TP}{2TP + FP + FN}, \quad iv$$

$$IoU = \frac{TP}{TP + FP + FN}, \quad v$$

$$Precision = \frac{TP}{TP + FP}, \quad vi$$

$$Recall = \frac{TP}{TP + FN}, \quad vii$$

where TP , FP , and FN denote the numbers of true positive, false positive, and false negative predictions, respectively.

Table 8 *Quantitative comparison of different intensity normalization methods based on the best validation Dice score and independent test-set segmentation metrics.*

Method	Best Validation Dice	TestDice	TestIoU	TestPrecision	TestRecall
No Normalization	0.4751	0.3242	0.1987	0.1987	1
Min-Max	0.4744	0.5702	0.416	0.4421	0.945
Min-Max (Non-zero)	0.5421	0.4973	0.3433	0.3433	1
Z-score	0.6906	0.5861	0.441	0.4724	0.914

As shown in , intensity normalization considerably influenced the segmentation performance of the proposed model. Among the evaluated methods, Z-score normalization achieved the highest best validation Dice score (0.6906) and also provided the best overall test performance, with a Dice score of 0.5861 and an IoU of 0.4410. These results indicate that Z-score normalization offers a better balance between detecting tumor regions and reducing false-positive predictions. Although the no normalization

and Min-Max (Non-zero) approaches achieved a Recall value of 1.0, their relatively low Precision values suggest that many normal slices were incorrectly identified as tumor-containing slices, resulting in lower Dice and IoU scores. Compared with conventional Min-Max normalization, Z-score normalization is less sensitive to variations in image intensity distributions across MRI acquisitions, thereby improving model generalization. Since excluding background pixels reduced the performance of Min-Max normalization, a similar experiment using Z-score normalization without background pixels was not considered necessary. Therefore, standard Z-score normalization was selected for all subsequent experiments.

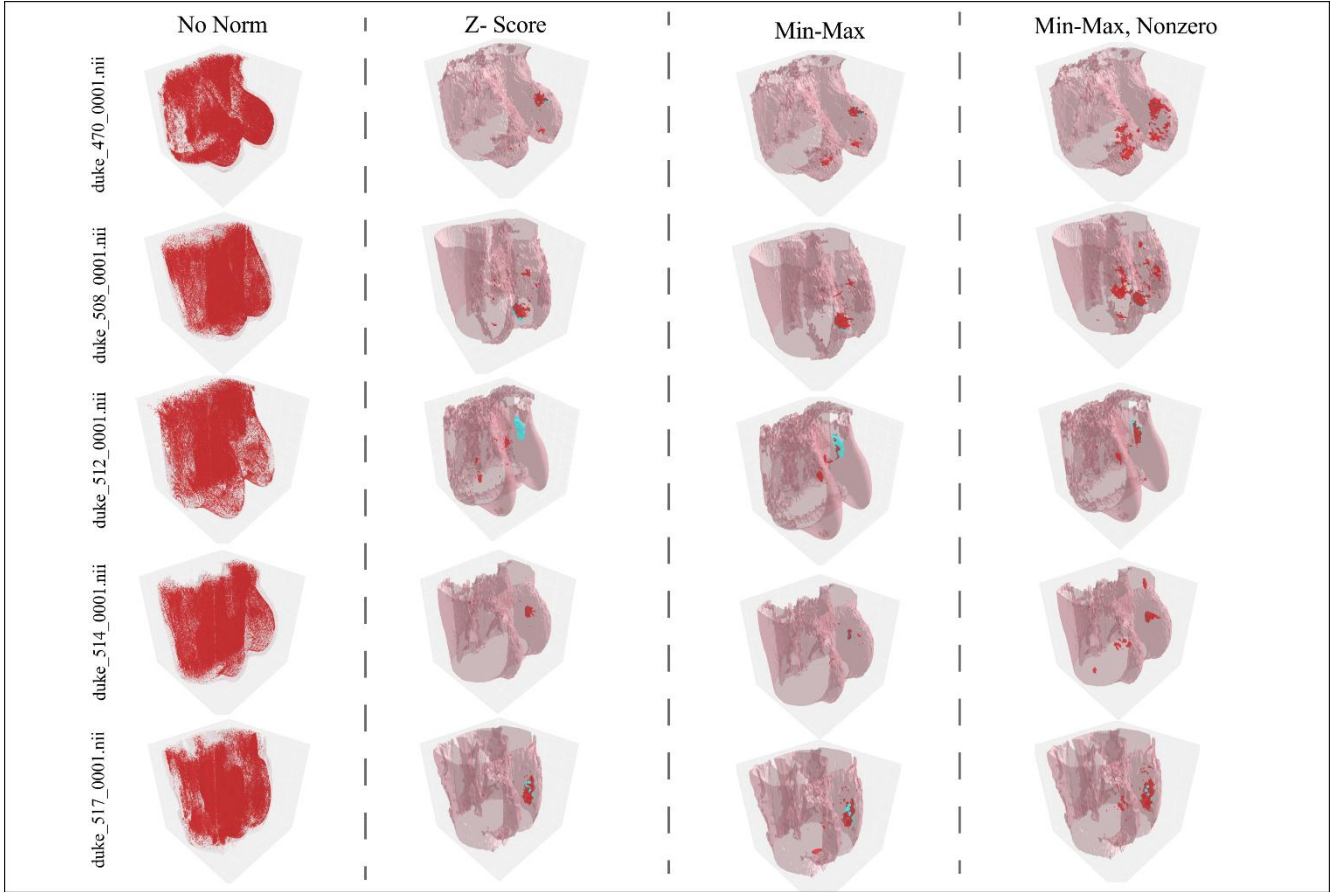


Figure 14 Qualitative comparison of segmentation results obtained using different intensity normalization methods for the first five representative test MRI volumes. The predicted tumor regions are overlaid on the corresponding breast MRI volumes, illustrating the effect of each normalization strategy on segmentation performance.

Figure 14 and Figure 15 present a qualitative comparison of the segmentation outputs generated using the four evaluated intensity normalization methods across ten representative test MRI volumes. The no normalization approach produces extensive false-positive predictions distributed throughout the

breast tissue, whereas the normalized methods generate more localized and anatomically meaningful tumor regions.

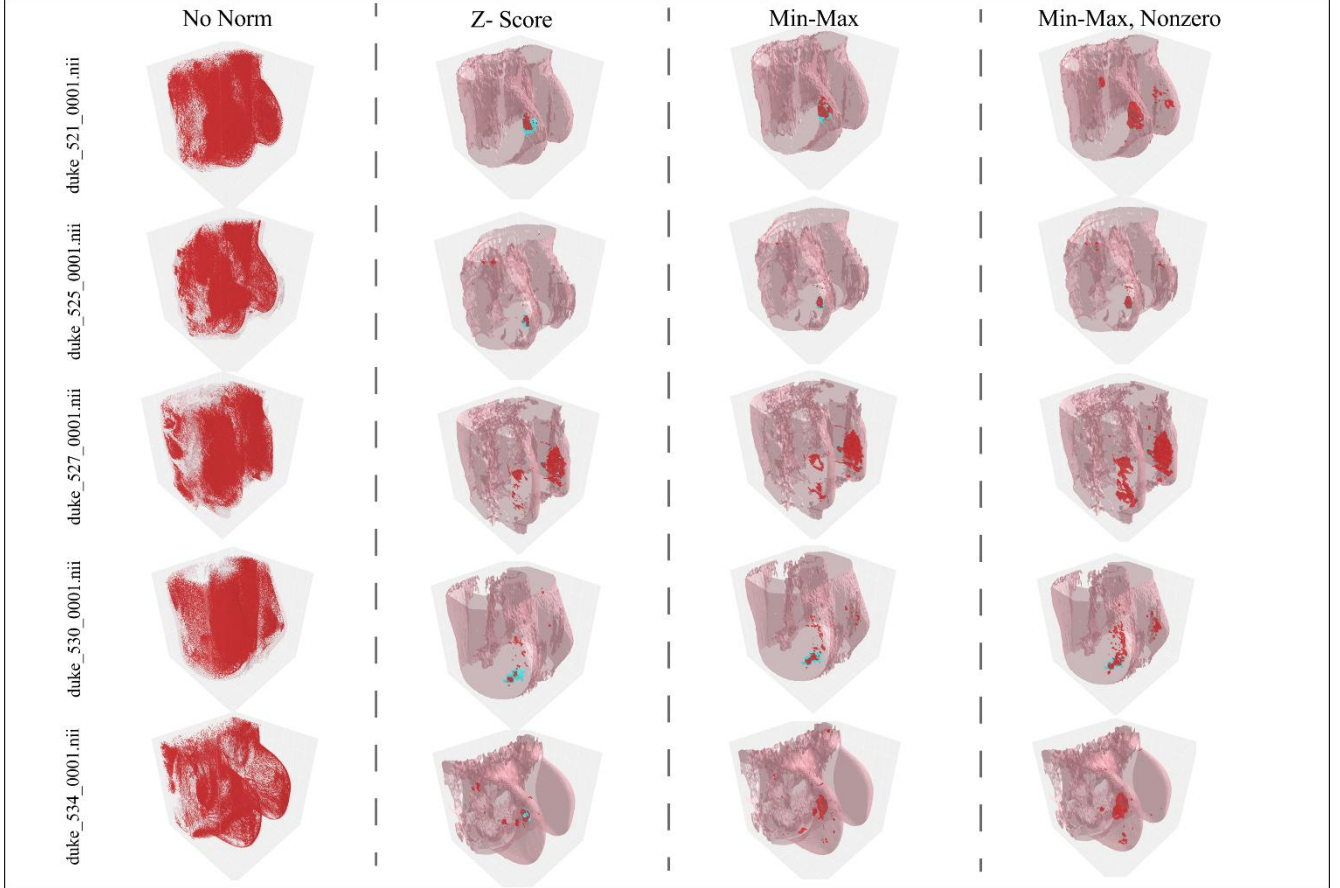


Figure 15 Qualitative comparison of segmentation results obtained using different intensity normalization methods for the remaining five representative test MRI volumes. The visualization further demonstrates the differences in segmentation quality produced by the evaluated normalization approaches.

Among the evaluated techniques, Z-score normalization consistently produces segmentations that show better agreement with the expert annotations while reducing spurious detections. In comparison, both Min-Max normalization methods occasionally exhibit fragmented or scattered false-positive regions. These qualitative observations support the quantitative results presented in Table 7 and further justify the selection of Z-score normalization as the preprocessing strategy for all subsequent segmentation experiments.

2.7 Evaluation of Post-processing Strategy

To establish an appropriate threshold for removing insignificant segmented regions, the ground-truth tumor volumes of all 40 annotated cases were calculated and arranged in descending order. The smallest expert-annotated tumor volume was found to be approximately **377.89 mm³**. Since any genuine tumor in the dataset occupied a volume greater than this value, a post-processing rule was implemented to automatically remove all predicted connected components with volumes smaller than 377.89 mm³. This approach was based on the assumption that such small isolated regions were more likely to represent false-positive detections rather than clinically relevant tumors.

The objective of this post-processing strategy was to suppress clinically insignificant false-positive components while preserving the actual tumor anatomy and maintaining the integrity of the primary lesion.

Table 9 illustrates the distribution of expert-annotated tumor volumes for all 40 cases in the dataset, arranged in descending order. A considerable variation in tumor size can be observed, ranging from approximately 141,607 mm³ to 377.89 mm³. The smallest annotated tumor volume was subsequently adopted as the lower threshold for removing insignificant connected components during post-processing.

Table 9 Distribution of ground-truth tumor volumes across the 40 annotated MRI cases arranged in descending order

File	Tumor Voxels	Tumor Volume (mm ³)
duke_090.nii.gz	303028	141607.41
duke_057.nii.gz	132410	128251.07
duke_046.nii.gz	62016	43452.05
duke_097.nii.gz	50613	30891.72
duke_091.nii.gz	45607	28895.22
duke_061.nii.gz	62088	27382.53
duke_517.nii.gz	58555	27363.28
duke_512.nii.gz	39184	23916.02
duke_086.nii.gz	38772	23664.55
duke_019.nii.gz	17175	21327.86
duke_034.nii.gz	31765	18843.6
duke_082.nii.gz	28454	18027.6
duke_527.nii.gz	25713	16603.58

duke_010.nii.gz	35072	15467.76
duke_508.nii.gz	30980	15314.99
duke_040.nii.gz	20734	13136.44
duke_005.nii.gz	19771	12067.26
duke_043.nii.gz	24513	11455.13
duke_530.nii.gz	18162	8009.97
duke_021.nii.gz	14478	7975.36
duke_060.nii.gz	16490	7705.93
duke_041.nii.gz	16450	7687.23
duke_525.nii.gz	13434	7400.25
duke_521.nii.gz	18733	7317.57
duke_099.nii.gz	17566	6861.74
duke_470.nii.gz	11133	6396.18
duke_001.nii.gz	7477	5310.91
duke_009.nii.gz	10673	5177.81
duke_064.nii.gz	11649	4838.76
duke_101.nii.gz	8734	4600.16
duke_022.nii.gz	7550	4506.21
duke_514.nii.gz	8175	4041.31
duke_534.nii.gz	6224	4019
duke_044.nii.gz	4472	2833.32
duke_048.nii.gz	4110	2367.25
duke_059.nii.gz	5055	2229.4
duke_069.nii.gz	3815	1682.52
duke_055.nii.gz	3527	1465.05
duke_002.nii.gz	1712	764
duke_051.nii.gz	686	377.89

Figure 16 and Figure 17 illustrates the effect of applying the minimum tumor volume threshold during the post-processing stage. The left column for each case shows the original predicted segmentation, while the corresponding right column shows the result after removing all connected components with volumes smaller than 377.89 mm³. The eliminated regions primarily consist of small isolated false-positive detections, whereas the main tumor structures are preserved. This observation demonstrates that the volume-based filtering strategy effectively suppresses insignificant segmented regions without substantially altering the clinically relevant tumor anatomy.

After removing connected components with volumes smaller than 377.89 mm^3 , the remaining segmented regions were further evaluated based on their spatial relationship with the main tumor component. Components identified as isolated false-positive regions were labeled as "Remove," whereas components corresponding to the primary lesion or anatomically plausible neighboring regions were labeled as "Keep."

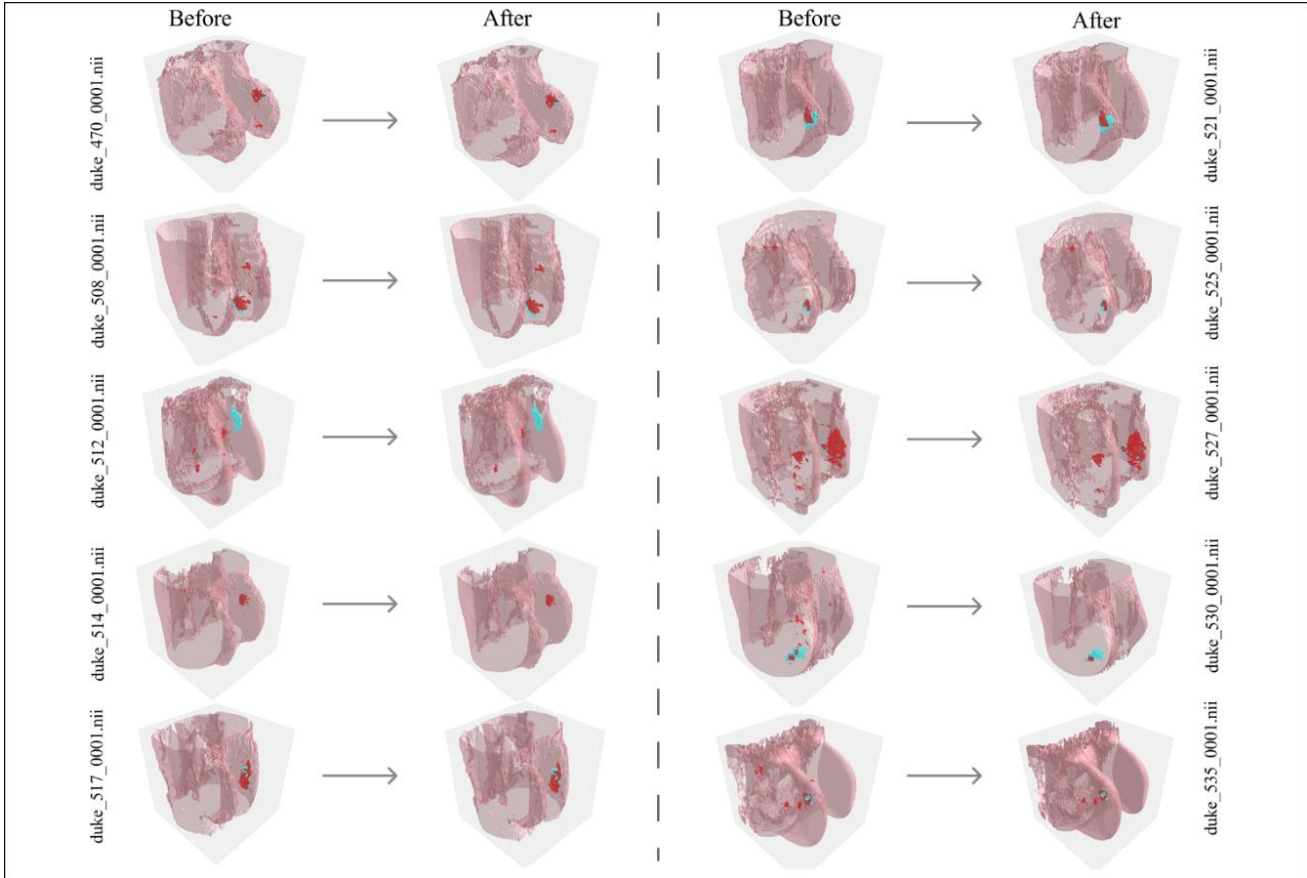


Figure 16 Illustration of the two-stage post-processing strategy. First being (a) Elimination of predicted tumor regions with volumes below 377.89 mm^3 .

In a few cases, components that satisfied the removal criteria overlapped with the expert annotation and were therefore retained, as indicated by "Couldn't Remove." This additional filtering step helped reduce false-positive detections while preserving clinically relevant tumor regions.

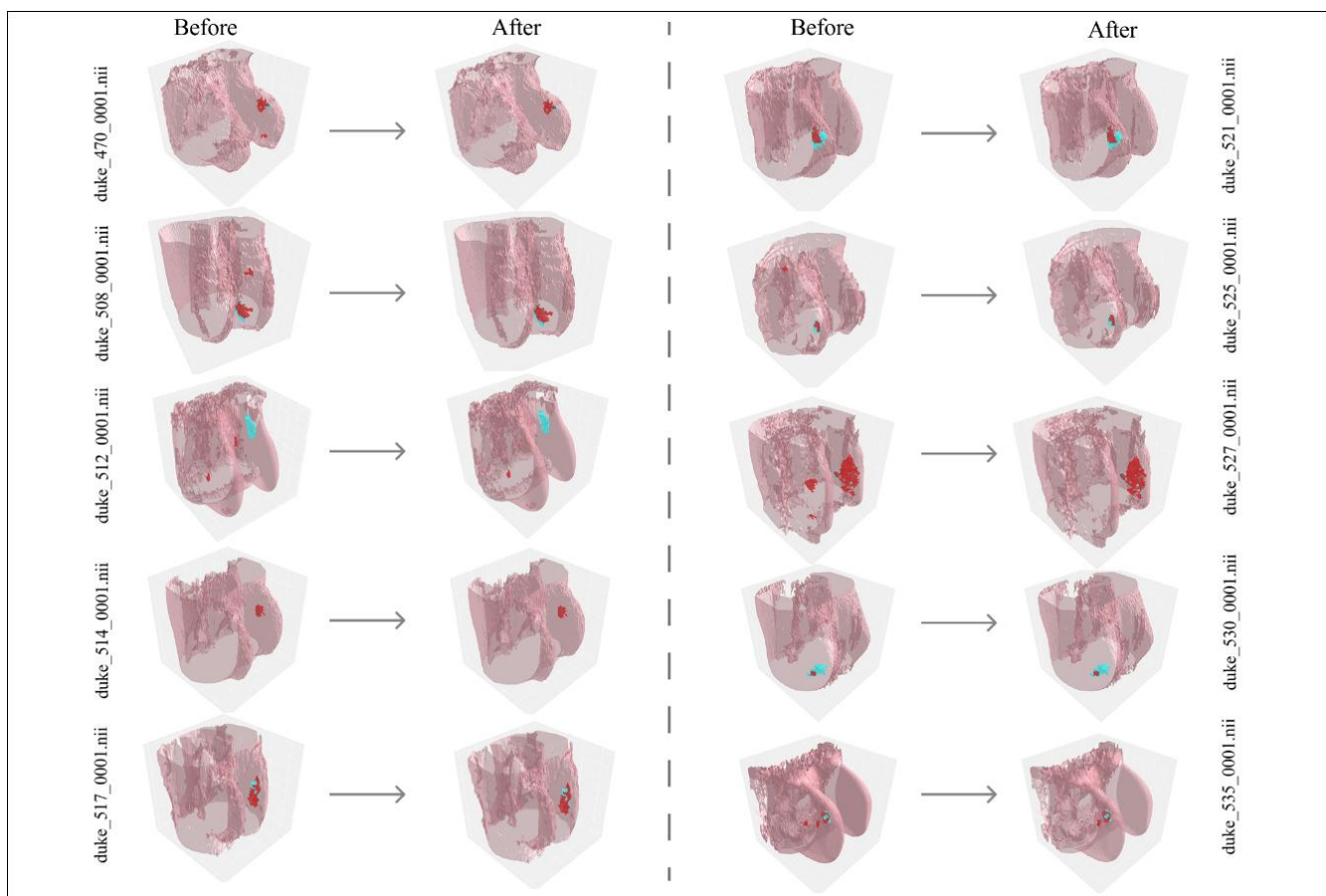


Figure 17 . Illustrates second stage of postprocessing (b) Removal of secondary tumor regions whose centroid distance from the primary tumor exceeds the median distance threshold

3 Conclusion

This work presented an automated patient-specific breast tumor segmentation framework based on a 2D U-Net architecture implemented using the MONAI framework. A complete segmentation pipeline was developed, incorporating volume-level dataset preparation, voxel spacing standardization, slice extraction, padding-based spatial standardization, intensity normalization, and a two-stage post-processing strategy to improve segmentation quality.

A comparative evaluation of different intensity normalization methods demonstrated that Z-score normalization achieved the highest validation Dice score of **0.6906**, while Min–Max normalization produced the best independent test performance with a **Dice score of 0.5702**, an **IoU of 0.4160**, a **Precision of 0.4421**, and a **Recall of 0.9450**. Furthermore, the proposed post-processing approach, consisting of the removal of tumor regions smaller than **377.89 mm³** followed by elimination of anatomically isolated predictions using a **median centroid-distance threshold of 63.69 mm**, effectively reduced false-positive detections while preserving clinically plausible tumor structures.

Overall, the proposed framework provides a computationally efficient and robust solution for automated breast MRI segmentation using slice-based deep learning, achieving reliable tumor localization while remaining suitable for systems with limited computational resources. The generated segmentation masks can serve as a valuable component for patient-specific image-guided breast cancer therapeutic planning.

4 Future work

- Increase or decrease the number of feature channels (e.g., from **(16, 32, 64, 128, 256)** to **(32, 64, 128, 256, 512)**) to study its effect on feature extraction.
- Experiment with different network depths by adding or removing one encoder-decoder level to balance segmentation accuracy and computational cost.
- Vary the number of residual convolutional units (**num_res_units**) from 2 to 1, 3, or 4 to investigate their influence on gradient flow and model performance.
- Introduce dropout layers or vary the dropout probability (e.g., 0.1–0.3) to reduce overfitting and improve model generalization. The current implementation does not use dropout, and future studies may investigate whether regularization improves segmentation performance.
- Evaluate different activation functions such as ReLU, Leaky ReLU, Mish, and GELU in place of the default PReLU activation used by the current MONAI 2D U-Net. Different activation functions may influence feature extraction capability and training convergence.
- Investigate the effect of using larger convolution kernels, such as 5×5 instead of the currently used 3×3 kernels, to capture additional contextual information around tumor boundaries. Larger receptive fields may help improve the delineation of irregular tumor regions.

5 References

- [1] H. Sung, J. Ferlay, R. L. Siegel, M. Laversanne, I. Soerjomataram and A. Jemal, "Global Cancer Statistics 2022: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries," *CA: A Cancer Journal for Clinicians*, pp. 229-263, 2024.
- [2] W. H. Organization, "Breast cancer," 2026. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/breast-cancer>.
- [3] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Springer*, Cham, 2015.
- [4] M. J. Cardoso, W. Li, R. Brown, N. Ma, E. Kerfoot and Y. Wang, "MONAI: An open-source framework for deep learning in healthcare," 2022.
- [5] S. K. Yoo, T. H. Kim, J. Chun, B. S. Choi, H. Kim and S. Yang, "Deep-learning-based automatic detection and segmentation of brain metastases with small volume for stereotactic ablative radiotherapy," *Cancers*, p. 2555, 2022.
- [6] L. Garrucho, K. Kushibar, C. A. Reidel, S. Joshi, R. Osuala and A. Tsirikoglou, "A large-scale multicenter breast cancer DCE-MRI benchmark dataset with expert segmentations," *Scientific Data*, p. 453, 2025.
- [7] A. Diaz-Pinto, S. Alle, V. Nath, Y. Tang, A. Ihsani and M. Asad, "MONAI Label: A framework for AI-assisted interactive labeling of 3D medical images," *Medical Image Analysis*, vol. 2024, p. 103207, 2024.

Appendix

- 2D_slicing.py

```
import os
import glob
import nibabel as nib
import numpy as np
from tqdm import tqdm
import scipy.ndimage as ndimage
from sklearn.model_selection import train_test_split

# =====
# Configuration
# =====
DATA_DIR = r"Data\Training_data"
TRAIN_IMAGES = os.path.join(DATA_DIR, "imagesTr")
TRAIN_LABELS = os.path.join(DATA_DIR, "labelsTr")

OUTPUT_DIR = r"Data\2d_slice_5{3}"

OUT_IMAGES_TRAIN = os.path.join(OUTPUT_DIR, "imagesTr")
OUT_LABELS_TRAIN = os.path.join(OUTPUT_DIR, "labelsTr")

OUT_IMAGES_VAL = os.path.join(OUTPUT_DIR, "imagesVal")
OUT_LABELS_VAL = os.path.join(OUTPUT_DIR, "labelsVal")

os.makedirs(OUT_IMAGES_TRAIN, exist_ok=True)
os.makedirs(OUT_LABELS_TRAIN, exist_ok=True)
os.makedirs(OUT_IMAGES_VAL, exist_ok=True)
os.makedirs(OUT_LABELS_VAL, exist_ok=True)

# =====
# Padding Function
# =====
def pad_or_crop_512(img):
    h, w = img.shape

    pad_h = max(0, 512 - h)
    pad_w = max(0, 512 - w)
```

```

img = np.pad(
    img,
    (
        (pad_h // 2, pad_h - pad_h // 2),
        (pad_w // 2, pad_w - pad_w // 2),
    ),
    mode="constant",
)

return img

# =====
# Load Volumes
# =====
images = sorted(glob.glob(os.path.join(TRAIN_IMAGES, "*.nii*")))
labels = sorted(glob.glob(os.path.join(TRAIN_LABELS, "*.nii*")))

data_pairs = list(zip(images, labels))

# =====
# Train/Validation Split
# =====
train_pairs, val_pairs = train_test_split(
    data_pairs,
    test_size=0.2,
    random_state=42,
)

print(f"Train volumes: {len(train_pairs)} | Val volumes: {len(val_pairs)}")

# =====
# Processing Function
# =====
def process(pairs, out_img_dir, out_lbl_dir):

    total_slices = 0

    for img_path, lbl_path in tqdm(pairs):

        base_name = os.path.basename(img_path).split(".")[0]

        img_obj = nib.load(img_path)
        lbl_obj = nib.load(lbl_path)

        img_data = img_obj.get_fdata()
        lbl_data = lbl_obj.get_fdata()

```

```

spacing = img_obj.header.get_zooms()[:3]

target_spacing = (
    1.0,
    1.0,
    spacing[2],
)

resize_factors = (
    spacing[0] / target_spacing[0],
    spacing[1] / target_spacing[1],
    1,
)

img_data = ndimage.zoom(
    img_data,
    resize_factors,
    order=1,
)

lbl_data = ndimage.zoom(
    lbl_data,
    resize_factors,
    order=0,
)

depth = img_data.shape[-1]

# =====
# Slice-wise Processing
# =====
for z in range(depth):

    img_slice = img_data[:, :, z]
    lbl_slice = lbl_data[:, :, z]

    img_slice = pad or crop 512(img_slice)
    lbl_slice = pad or crop 512(lbl_slice)

    slice_name = (
        f'{base_name} slice {z:05d}.npy'
    )

    np.save(
        os.path.join(
            out_img_dir,

```



```

        slice_name,
    ),
    img_slice.astype(np.float32),
)

np.save(
    os.path.join(
        out_lbl_dir,
        slice_name,
    ),
    lbl_slice.astype(np.uint8),
)

total_slices += 1

return total_slices

# =====
# Run
# =====
print("\nProcessing TRAIN...")
train_slices = process(
    train_pairs,
    OUT_IMAGES_TRAIN,
    OUT_LABELS_TRAIN,
)

print("\nProcessing VAL...")
val_slices = process(
    val_pairs,
    OUT_IMAGES_VAL,
    OUT_LABELS_VAL,
)

print("\nDone!")
print(f"Train slices: {train_slices}")
print(f"Val slices: {val_slices}")

```

- **train_2D_Unet.py**

```

import os
import glob

```

```

import torch
import time
from datetime import datetime
from monai.metrics import DiceMetric
from monai.transforms import AsDiscrete
from monai.data import decollate_batch
from monai.losses import DiceCELoss
from monai.networks.nets import UNet
from monai.data import Dataset, DataLoader
from monai.transforms import (
    Compose,
    LoadImaged,
    EnsureChannelFirstd,
    RandRotate90d,
    RandFlipd,
    EnsureTyped,
    NormalizeIntensityd, # Z- score normalization
)

# =====
# CONFIGURATION
# =====
DATA_DIR = r"data\2d slice 5{3}"

TRAIN_IMAGES = os.path.join(DATA_DIR, "imagesTr")
TRAIN_LABELS = os.path.join(DATA_DIR, "labelsTr")

VAL_IMAGES = os.path.join(DATA_DIR, "imagesVal")
VAL_LABELS = os.path.join(DATA_DIR, "labelsVal")

NUM_CLASSES = 4
BATCH_SIZE = 8
NUM_WORKERS = 0
MAX_EPOCHS = 50
LEARNING_RATE = 1e-4

DEVICE = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

# =====
# START TIME
# =====
start_time = time.time()
print("Training started at:", datetime.now())

# =====
# LOAD FILE PATHS
# =====

```

```

train_images = sorted(glob.glob(os.path.join(TRAIN_IMAGES, "*.npz")))
train_labels = sorted(glob.glob(os.path.join(TRAIN_LABELS, "*.npz")))

val_images = sorted(glob.glob(os.path.join(VAL_IMAGES, "*.npz")))
val_labels = sorted(glob.glob(os.path.join(VAL_LABELS, "*.npz")))

train_files = [{"image": img, "label": lbl} for img, lbl in zip(train_images, train_labels)]
val_files = [{"image": img, "label": lbl} for img, lbl in zip(val_images, val_labels)]

print(f"Training on {len(train_files)} slices | Validating on {len(val_files)} slices")

# =====
# TRANSFORMS
# =====
train_transforms = Compose([
    LoadImage(keys=["image", "label"], reader="NumpyReader"),
    EnsureChannelFirstd(keys=["image", "label"]),
    NormalizeIntensityd(keys=["image"], nonzero=False, channel_wise=True), # z score normalization
    RandRotate90d(keys=["image", "label"], prob=0.5, spatial_axes=(0, 1)),
    RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=0),
    RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=1),

    EnsureTyped(keys=["image", "label"]),
])

val_transforms = Compose([
    LoadImage(keys=["image", "label"], reader="NumpyReader"),
    EnsureChannelFirstd(keys=["image", "label"]),

    NormalizeIntensityd(keys=["image"], nonzero=False, channel_wise=True),

    EnsureTyped(keys=["image", "label"]),
])

# =====
# DATASETS & LOADERS
# =====
train_ds = Dataset(data=train_files, transform=train_transforms)
train_loader = DataLoader(
    train_ds,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS
)

val_ds = Dataset(data=val_files, transform=val_transforms)
val_loader = DataLoader(

```

```

    val_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS
)

# =====
# MODEL
# =====
model = UNet(
    spatial_dims=2,
    in_channels=1,
    out_channels=NUM_CLASSES,
    channels=(16, 32, 64, 128, 256),
    strides=(2, 2, 2, 2),
    num_res_units=2,
).to(DEVICE)

# =====
# LOSS, OPTIMIZER, METRIC
# =====

# Dice + Cross Entropy combined
loss_function = DiceCELoss(
    to_onehot_y=True,
    softmax=True,
    lambda_dice=0.5,
    lambda_ce=0.5
)

optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
dice_metric = DiceMetric(include_background=True, reduction="mean")

post_pred = AsDiscrete(argmax=True, to_onehot=NUM_CLASSES)
post_label = AsDiscrete(to_onehot=NUM_CLASSES)
# =====
# TRAINING LOOP
# =====
best_metric = -1

for epoch in range(MAX_EPOCHS):
    print(f"\nEpoch {epoch+1}/{MAX_EPOCHS}")

    epoch_start = time.time()

    # ----- TRAIN -----
    model.train()

```

```

epoch_loss = 0
step = 0

for batch_data in train_loader:
    step += 1

    inputs = batch_data["image"].to(DEVICE)
    labels = batch_data["label"].to(DEVICE)

    optimizer.zero_grad()
    outputs = model(inputs)

    loss = loss_function(outputs, labels)
    loss.backward()
    optimizer.step()

    epoch_loss += loss.item()

print(f"Average Train Loss: {epoch_loss/step:.4f}")

# ----- VALIDATION -----
model.eval()
dice_metric.reset()

with torch.no_grad():
    for val_data in val_loader:
        val_inputs = val_data["image"].to(DEVICE)
        val_labels = val_data["label"].to(DEVICE)

        val_outputs = model(val_inputs)

        # ☒ Proper formatting
        #  CORRECT MONAI WAY
        val_outputs = [post_pred(i) for i in decollate_batch(val_outputs)]
        val_labels = [post_label(i) for i in decollate_batch(val_labels)]

        dice_metric(y_pred=val_outputs, y=val_labels)
metric = dice_metric.aggregate().item()
print(f"Validation Mean Dice: {metric:.4f}")

if metric > best_metric:
    best_metric = metric
    torch.save(model.state_dict(), "best_model_8{a}.pth")
    print("--> Saved new best model!")

epoch_end = time.time()
print(f"Epoch Time: {epoch_end - epoch_start:.2f} sec")

```

```

# =====
# TOTAL TIME
# =====
end_time = time.time()

print("\nTraining complete!")
print(f"Best Validation Dice: {best_metric:.4f}")

total_time = end_time - start_time

print("\nTotal Training Time:")
print(f"{total_time:.2f} seconds")
print(f"{total_time/60:.2f} minutes")
print(f"{total_time/3600:.2f} hours")

print("Training ended at:", datetime.now())

```

- **Inference.py**

```

import os
import glob
import torch
import numpy as np
import nibabel as nib
from tqdm import tqdm
from monai.networks.nets import UNet
import scipy.ndimage as ndimage

# =====
# CONFIGURATION
# =====
INPUT_DIR = r"Data\Testing_data\imagesTs"
OUTPUT_DIR = r"Data\Inference 8{a}"

MODEL_WEIGHTS = "best_model_8{a}.pth"

NUM_CLASSES = 4
DEVICE = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# PADDING FUNCTION (NO CROP) only till 512 if the image size is more than 512
# then that constraint is not added)

```

```

# =====
def pad_to_512(img):
    h, w = img.shape

    pad_h = max(0, 512 - h)
    pad_w = max(0, 512 - w)

    img = np.pad(
        img,
        ((pad_h // 2, pad_h - pad_h // 2),
         (pad_w // 2, pad_w - pad_w // 2)),
        mode='constant'
    )

    return img

# =====
# REMOVE PADDING FUNCTION
# =====
def remove_padding(pred, original_shape):
    h, w = original_shape

    pad_h = pred.shape[0] - h
    pad_w = pred.shape[1] - w

    start_h = pad_h // 2
    start_w = pad_w // 2

    return pred[start_h:start_h + h, start_w:start_w + w]

# =====
# LOAD MODEL
# =====
print("Loading model...")

model = UNet(
    spatial_dims=2,
    in_channels=1,
    out_channels=NUM_CLASSES,
    channels=(16, 32, 64, 128, 256),
    strides=(2, 2, 2, 2),
    num_res_units=2,
).to(DEVICE)

model.load_state_dict(torch.load(MODEL_WEIGHTS, map_location=DEVICE))
model.eval()

```

```

print("Model loaded successfully")

# =====
# LOAD FILES
# =====
input_files = sorted(glob.glob(os.path.join(INPUT_DIR, "*.nii*")))

print(f"Found {len(input_files)} volumes")

# =====
# INFERENCE
# =====
with torch.no_grad():

    for f_path in tqdm(input_files, desc="Processing"):

        fname = os.path.basename(f_path)

        if fname.endswith(".nii.gz"):
            out_name = fname[:-7] + ".nii"
        else:
            out_name = fname

        out_path = os.path.join(OUTPUT_DIR, out_name)

# =====
# Load volume
# =====
img_obj = nib.load(f_path)
img_data = img_obj.get_fdata()
affine = img_obj.affine

original_shape = img_data.shape

# =====
# Resample (same as training)
# =====
spacing = img_obj.header.get_zooms()[:3]

target_spacing = (1.0, 1.0, spacing[2])

resize_factors = (
    spacing[0] / target_spacing[0],
    spacing[1] / target_spacing[1],
    1
)

img_data = ndimage.zoom(img_data, resize_factors, order=1)

```



```

# =====
# Prepare output
# =====
pred_volume_resampled = np.zeros(img_data.shape, dtype=np.uint8)

depth = img_data.shape[-1]

# =====
# Slice-wise inference
# =====
for z in range(depth):

    slice_data = img_data[:, :, z]
    original_2d_shape = slice_data.shape
    # =====
    # Padding ONLY (NO CROP)
    # =====
    slice_padded = pad_to_512(slice_data)

    # =====
    # Convert to tensor
    # =====
    slice_tensor = torch.tensor(
        slice_padded,
        dtype=torch.float32
    ).unsqueeze(0).unsqueeze(0).to(DEVICE)

    # =====
    # Z-score normalization
    # =====
    mean = slice_tensor.mean()
    std = slice_tensor.std()

    if std > 1e-6:
        slice_tensor = (slice_tensor - mean) / std

    # =====
    # Predict
    # =====
    output = model(slice_tensor)
    pred_slice = torch.argmax(output, dim=1).squeeze().cpu().numpy()

    # =====
    # Remove padding
    # =====
    pred_restored = remove_padding(pred_slice, original_2d_shape)

```

```

pred_volume_resampled[:, :, z] = pred_restored

# =====
# Restore original resolution
# =====
pred_volume_original = ndimage.zoom(
    pred_volume_resampled,
    (
        original_shape[0] / pred_volume_resampled.shape[0],
        original_shape[1] / pred_volume_resampled.shape[1],
        1
    ),
    order=0
)

# =====
# Save output
# =====
pred_nifti = nib.Nifti1Image(pred_volume_original, affine)
nib.save(pred_nifti, out_path)

print("\n✓ Inference complete!")
print(f"Saved in: {OUTPUT_DIR}")

```

- **3D_view.py**

```

import os
import SimpleITK as sitk
import numpy as np
from skimage import measure
from skimage.measure import label
from scipy.ndimage import binary_fill_holes, gaussian_filter
import plotly.graph_objects as go
import plotly.io as pio

# =====
# 💧 FORCE BROWSER POPUP
# =====
pio.renderers.default = "browser"

# =====
# 📁 PATHS
# =====
pred_path = r>Data\Inference_7{b}\duke_534_0001.nii"
gt_path = r>Data\Expert segmentation\Testing_dataset\duke_534.nii.gz"

```

```

img_path = r>Data\Testing_data\imagesTs\duke_534_0001.nii"

# =====
# 🧠 EXTRACT NAMES
# =====
file_name = os.path.basename(pred_path)

patient_id = file_name.replace("_0001.nii", "") \
    .replace(".nii.gz", "") \
    .replace(".nii", "")

inference_name = os.path.basename(os.path.dirname(pred_path))

# =====
# LOAD DATA
# =====
pred_np = sitk.GetArrayFromImage(sitk.ReadImage(pred_path))
gt_np = sitk.GetArrayFromImage(sitk.ReadImage(gt_path))
img_np = sitk.GetArrayFromImage(sitk.ReadImage(img_path))

# =====
# EXTRACT TUMOR
# =====
pred_tumor = (pred_np == 1)
gt_tumor = (gt_np == 1)

if np.sum(pred_tumor) == 0 or np.sum(gt_tumor) == 0:
    print("⚠ Tumor not found → exiting")
    exit()

# =====
# 💧 SMOOTH BREAST SURFACE
# =====
tissue = img_np > np.percentile(img_np, 60)

tissue = gaussian_filter(tissue.astype(float), sigma=2)

tissue = tissue > 0.3

labels = label(tissue)
largest_cc = np.argmax(np.bincount(labels.flat)[1:]) + 1
body = (labels == largest_cc)

body = binary_fill_holes(body)

body_smooth = gaussian_filter(body.astype(float), sigma=1)

```

```

# =====
# CREATE SURFACES
# =====

verts_pred, faces_pred, _, _ = measure.marching_cubes(pred_tumor, level=0.5)
verts_gt, faces_gt, _, _ = measure.marching_cubes(gt_tumor, level=0.5)
verts_body, faces_body, _, _ = measure.marching_cubes(body_smooth, level=0.5)

# =====
# MESHES
# =====

mesh_body = go.Mesh3d(
    x=verts_body[:, 2],
    y=verts_body[:, 1],
    z=verts_body[:, 0],
    i=faces_body[:, 0],
    j=faces_body[:, 1],
    k=faces_body[:, 2],
    color='lightpink',
    opacity=0.1,
    name='Breast Surface',
    lighting=dict(
        ambient=0.6,
        diffuse=0.8,
        roughness=0.4,
        specular=0.3
    )
)

# =====
# GROUND TRUTH (PURPLE)
# =====

mesh_gt = go.Mesh3d(
    x=verts_gt[:, 2],
    y=verts_gt[:, 1],
    z=verts_gt[:, 0],
    i=faces_gt[:, 0],
    j=faces_gt[:, 1],
    k=faces_gt[:, 2],
    color="#55FDFF", # cyan
    opacity=0.6,
    name='Ground Truth'
)

# =====
# PREDICTION (RED)
# =====

mesh_pred = go.Mesh3d(

```

```

x=verts_pred[:, 2],
y=verts_pred[:, 1],
z=verts_pred[:, 0],
i=faces_pred[:, 0],
j=faces_pred[:, 1],
k=faces_pred[:, 2],
color='#E5383B',
opacity=0.6,
name=inference_name
)

# =====
# FIGURE
# =====
fig = go.Figure(data=[mesh_body, mesh_gt, mesh_pred])

# =====
# 📷 ISOMETRIC CAMERA
# =====
iso_camera = dict(
    eye=dict(x=1.8, y=1.8, z=1.8),
    up=dict(x=0, y=0, z=1),
    center=dict(x=0, y=0, z=0)
)

# =====
# 🎛️ OPACITY SLIDER
# =====
steps = []

for opacity in np.linspace(0, 1, 8):
    step = dict(
        method="restyle",
        args=[{"opacity": opacity}, [0]],
        label=f"{opacity:.2f}"
    )
    steps.append(step)

sliders = [
    dict(
        active=2,
        currentvalue={"prefix": "Breast Opacity: "},
        pad={"t": 50},
        steps=steps
    )
]

```

```

# =====
# LAYOUT
# =====
fig.update_layout(

    title=f"{{patient_id}} | {{inference_name}}<br>3D Tumor + Breast Surface",

    scene=dict(

        xaxis=dict(
            showticklabels=False,
            title="",
            showgrid=True,
            gridcolor="white",
            zeroline=False,
            showbackground=True,
            backgroundcolor="rgb(240,240,240)"
        ),

        yaxis=dict(
            showticklabels=False,
            title="",
            showgrid=True,
            gridcolor="white",
            zeroline=False,
            showbackground=True,
            backgroundcolor="rgb(240,240,240)"
        ),

        zaxis=dict(
            showticklabels=False,
            title="",
            showgrid=True,
            gridcolor="white",
            zeroline=False,
            showbackground=True,
            backgroundcolor="rgb(240,240,240)"
        ),

        aspectmode="cube",

        camera=iso_camera

    ),
    sliders=sliders,

    updatemenus=[

```

```

dict(
    type="buttons",

    buttons=[

        dict(
            label="Isometric View",

            method="relayout",

            args=[{
                "scene.camera": iso_camera
            }]
        )

    ],

    direction="left",

    x=0.25,
    y=1.15,

    showactive=False
)

],

margin=dict(
    l=0,
    r=0,
    b=0,
    t=40
)
)

# =====
# SINGLE BROWSER OPEN
# =====
fig.show()

```